

# **Data Engineering Practicals (Sem 5)**

Practical Journal — Practicals 1 to 10

Name: Rishikesh Singh

Class: TY B.Sc Data Science

Subject: Data Engineering

Roll No: 53

# Practical 1: Multi-Format Data Parsing and Anomaly Inspection

## Aim

To write programs to parse text, CSV, HTML, XML, and JSON files, extract relevant data, and perform quality inspection checks to identify missing values, duplicate records, and data anomalies.

## Code

```
import pandas as pd
import numpy as np
import json
import xml.etree.ElementTree as ET
from bs4 import BeautifulSoup

COLS = ["id", "name", "age", "salary", "city", "email"]

# =====
# SETUP: sample files in 5 formats (same records, deliberate problems)
# =====
base = [
    (1, "Asha Rao", 28, 55000, "Mumbai", "asha@mail.com"),
    (2, "Rohan Das", 34, 62000, "Pune", "rohan@mail.com"),
    (3, "Meera Iyer", 29, None, "Chennai", "meera@mail.com"), # missing salary
    (4, "Karan Shah", 250, 58000, "Delhi", "karan@mail.com"), # anomaly: age 250
    (5, "Neha Gupta", 31, 61000, "Jaipur", None), # missing email
    (6, "Vikram Singh", 45, -5000, "Kolkata", "vikram@mail.com"), # anomaly: negative salary
    (2, "Rohan Das", 34, 62000, "Pune", "rohan@mail.com"), # duplicate row
    (7, "Priya Nair", 27, 54000, "Kochi", "priya-at-mail.com"), # anomaly: invalid email
]
val = lambda v: "None" if v is None else str(v)

# TXT
with open("sample.txt", "w", encoding="utf-8") as f:
    f.write("# Employee records (text format)\n")
    for r in base:
        f.write(", ".join(f"{c}: {val(v)}" for c, v in zip(COLS, r)) + "\n")

# CSV
pd.DataFrame(base, columns=COLS, dtype=object).to_csv("sample.csv", index=False)

# HTML
head = "".join("<th>" + c + "</th>" for c in COLS)
body = ""
for r in base:
    cells = "".join("<td>" + (" " if v is None else str(v)) + "</td>" for v in r)
    body += "<tr>" + cells + "</tr>"
with open("sample.html", "w", encoding="utf-8") as f:
    f.write("<html><body><h2>Employees</h2><table id='emp'><thead><tr>" + head +
        "</tr></thead><tbody>" + body + "</tbody></table></body></html>")

# XML (a missing value = the element is absent)
root = ET.Element("employees")
for r in base:
    e = ET.SubElement(root, "employee", id=str(r[0]))
    for c, v in zip(COLS[1:], r[1:]):
        if v is not None:
            ET.SubElement(e, c).text = str(v)
ET.ElementTree(root).write("sample.xml", encoding="utf-8", xml_declaration=True)

# JSON (nested address, to show flattening)
json_data = [{"id": r[0], "name": r[1], "age": r[2], "salary": r[3],
    "address": {"city": r[4]}, "email": r[5]} for r in base]
with open("sample.json", "w", encoding="utf-8") as f:
    json.dump(json_data, f, indent=2)

print("Files created: sample.txt, sample.csv, sample.html, sample.xml, sample.json")

# =====
# QUALITY INSPECTION
# =====
EMAIL_RE = r"^[w.-]+@[w.-]+\.[w.-]+$"
```

```
def inspect_data(df, name):
    d = df.copy()
    for c in ["id", "age", "salary"]:
        # text formats give strings
        d[c] = pd.to_numeric(d[c], errors="coerce")

    line = "-" * 20
    print(f"\n{line} DATA REPORT: {name} {line}")
    print(f"Shape: {d.shape[0]} rows x {d.shape[1]} columns")

    print("\nMissing values:")
    miss = d.isna().sum()
    print(miss[miss > 0].to_string() if miss.any() else "None")

    dup_count = int(d.duplicated().sum())
    print(f"\nDuplicate rows: {dup_count}")
    if dup_count:
        print(d[d.duplicated(keep=False)].to_string(index=False))

    print("\nAnomalies:")
    checks = {
        "age outside 18-65": d["age"].notna() & ~d["age"].between(18, 65),
        "salary <= 0": d["salary"].notna() & (d["salary"] <= 0),
        "invalid email": d["email"].notna() & ~d["email"].astype(str).str.match(EMAIL_RE),
    }
    any_bad = pd.Series(False, index=d.index)
    for label, mask in checks.items():
        if mask.any():
            print(f" - {label}: {int(mask.sum())} row(s)")
            print(d[mask].to_string(index=False))
            any_bad |= mask
    if not any_bad.any():
        print("None")

    return {"format": name, "rows": len(d), "missing_cells": int(d.isna().sum().sum()),
            "duplicates": dup_count, "anomaly_rows": int(any_bad.sum())}
```

## Output

Files created: sample.txt, sample.csv, sample.html, sample.xml, sample.json

## Parsers

### Task 1: Text file

#### Code

```
def parse_text_file(filepath):
    records = []
    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            record = {}
            for part in line.split(","):
                if ":" in part:
                    k, v = part.split(":", 1)
                    record[k.strip()] = v.strip() if v.strip() != "None" else None
            if record:
                records.append(record)
    return pd.DataFrame(records)
```

### Task 2: CSV file

#### Code

```
def parse_csv_file(filepath):
    return pd.read_csv(filepath)
```

### Task 3: HTML file (table)

## Code

```
def parse_html_file(filepath):
    with open(filepath, "r", encoding="utf-8") as f:
        soup = BeautifulSoup(f.read(), "html.parser")
        table = soup.find("table")
        headers = [th.get_text(strip=True) for th in table.find_all("th")]
        records = []
        for tr in table.find("tbody").find_all("tr"):
            cells = [td.get_text(strip=True) or None for td in tr.find_all("td")]
            records.append(dict(zip(headers, cells)))
    return pd.DataFrame(records)
```

## Task 4: XML file

### Code

```
def parse_xml_file(filepath):
    root = ET.parse(filepath).getroot()
    records = []
    for emp in root.findall("employee"):
        rec = {"id": emp.get("id")}
        for c in COLS[1:]:
            el = emp.find(c)
            rec[c] = el.text.strip() if el is not None and el.text else None
        records.append(rec)
    return pd.DataFrame(records)
```

## Task 5: JSON file (flatten nested fields)

### Code

```
def parse_json_file(filepath):
    with open(filepath, "r", encoding="utf-8") as f:
        data = json.load(f)
    df = pd.json_normalize(data).rename(columns={"address.city": "city"})
    return df[COLS]
```

### Code

```
summary = []
df_txt = parse_text_file("sample.txt"); summary.append(inspect_data(df_txt, "TXT Document"))
df_csv = parse_csv_file("sample.csv"); summary.append(inspect_data(df_csv, "CSV Document"))
df_html = parse_html_file("sample.html"); summary.append(inspect_data(df_html, "HTML Document"))
df_xml = parse_xml_file("sample.xml"); summary.append(inspect_data(df_xml, "XML Document"))
df_json = parse_json_file("sample.json"); summary.append(inspect_data(df_json, "JSON Document"))

print("\n" + "=" * 60)
print("SUMMARY ACROSS ALL FORMATS")
print("=" * 60)
print(pd.DataFrame(summary).to_string(index=False))
```

## Output

```
----- DATA REPORT: TXT Document -----
Shape: 8 rows x 6 columns
Missing values:
salary    1
email     1
Duplicate rows: 1
id  name age salary city    email
2 Rohan Das  34 62000.0 Pune rohan@mail.com
2 Rohan Das  34 62000.0 Pune rohan@mail.com
Anomalies:
- age outside 18-65: 1 row(s)
id  name age salary city    email
4 Karan Shah  250 58000.0 Delhi karan@mail.com
- salary <= 0: 1 row(s)
id  name age salary city    email
6 Vikram Singh  45 -5000.0 Kolkata vikram@mail.com
- invalid email: 1 row(s)
id  name age salary city    email
7 Priya Nair  27 54000.0 Kochi priya-at-mail.com
```

```
----- DATA REPORT: CSV Document -----
Shape: 8 rows x 6 columns
Missing values:
salary 1
email 1
Duplicate rows: 1
id name age salary city email
2 Rohan Das 34 62000.0 Pune rohan@mail.com
2 Rohan Das 34 62000.0 Pune rohan@mail.com
Anomalies:
- age outside 18-65: 1 row(s)
id name age salary city email
4 Karan Shah 250 58000.0 Delhi karan@mail.com
- salary <= 0: 1 row(s)
id name age salary city email
6 Vikram Singh 45 -5000.0 Kolkata vikram@mail.com
- invalid email: 1 row(s)
id name age salary city email
7 Priya Nair 27 54000.0 Kochi priya-at-mail.com
----- DATA REPORT: HTML Document -----
Shape: 8 rows x 6 columns
Missing values:
salary 1
email 1
Duplicate rows: 1
id name age salary city email
2 Rohan Das 34 62000.0 Pune rohan@mail.com
2 Rohan Das 34 62000.0 Pune rohan@mail.com
Anomalies:
- age outside 18-65: 1 row(s)
id name age salary city email
4 Karan Shah 250 58000.0 Delhi karan@mail.com
- salary <= 0: 1 row(s)
id name age salary city email
6 Vikram Singh 45 -5000.0 Kolkata vikram@mail.com
- invalid email: 1 row(s)
id name age salary city email
7 Priya Nair 27 54000.0 Kochi priya-at-mail.com
----- DATA REPORT: XML Document -----
Shape: 8 rows x 6 columns
Missing values:
salary 1
email 1
Duplicate rows: 1
id name age salary city email
2 Rohan Das 34 62000.0 Pune rohan@mail.com
2 Rohan Das 34 62000.0 Pune rohan@mail.com
Anomalies:
- age outside 18-65: 1 row(s)
id name age salary city email
4 Karan Shah 250 58000.0 Delhi karan@mail.com
- salary <= 0: 1 row(s)
id name age salary city email
6 Vikram Singh 45 -5000.0 Kolkata vikram@mail.com
- invalid email: 1 row(s)
id name age salary city email
7 Priya Nair 27 54000.0 Kochi priya-at-mail.com
----- DATA REPORT: JSON Document -----
Shape: 8 rows x 6 columns
Missing values:
salary 1
email 1
Duplicate rows: 1
id name age salary city email
2 Rohan Das 34 62000.0 Pune rohan@mail.com
2 Rohan Das 34 62000.0 Pune rohan@mail.com
Anomalies:
- age outside 18-65: 1 row(s)
id name age salary city email
4 Karan Shah 250 58000.0 Delhi karan@mail.com
- salary <= 0: 1 row(s)
id name age salary city email
6 Vikram Singh 45 -5000.0 Kolkata vikram@mail.com
- invalid email: 1 row(s)
```

| id | name       | age | salary  | city  | email             |
|----|------------|-----|---------|-------|-------------------|
| 7  | Priya Nair | 27  | 54000.0 | Kochi | priya-at-mail.com |

SUMMARY ACROSS ALL FORMATS

|               | format | rows | missing_cells | duplicates | anomaly_rows |
|---------------|--------|------|---------------|------------|--------------|
| TXT Document  |        | 8    | 2             | 1          | 3            |
| CSV Document  |        | 8    | 2             | 1          | 3            |
| HTML Document |        | 8    | 2             | 1          | 3            |
| XML Document  |        | 8    | 2             | 1          | 3            |
| JSON Document |        | 8    | 2             | 1          | 3            |

## Practical 2: ETL Pipeline Implementation & Data Transformation (MySQL Workbench)

### Aim

To implement an automated end-to-end relational ETL pipeline in MySQL Workbench, performing staging table population, stored procedure-based transformations, price discount categorisation, and multi-dimensional aggregate reporting.

### Steps 1–6: Database Setup, Staging Load, Target Tables, Validation & Transformation

#### Code

```
-- =====
-- PRACTICAL 2: ETL PIPELINE IMPLEMENTATION & DATA TRANSFORMATION
-- SINGLE-BLOCK VERSION
-- =====

-- =====
-- STEP 1: CREATE DATABASE
-- =====

DROP DATABASE IF EXISTS ETL_Lab_Practical2;

CREATE DATABASE ETL_Lab_Practical2;

USE ETL_Lab_Practical2;

-- =====
-- STEP 2: CREATE STAGING TABLES
-- =====

CREATE TABLE stg_Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    UnitPrice DECIMAL(10,2),
    UnitsInStock INT
);

CREATE TABLE stg_Orders (
    OrderID INT PRIMARY KEY,
    ProductID INT,
    CustomerName VARCHAR(100),
    Quantity INT,
    Discount DECIMAL(5,2),
    OrderDate DATE
);

-- =====
-- STEP 3: LOAD SOURCE DATA
```

```
-- =====  
  
INSERT INTO stg_Products  
  (ProductID, ProductName, Category, UnitPrice, UnitsInStock)  
VALUES  
  (101, 'LAPTOP PRO 15', 'ELECTRONICS', 1200.00, 15),  
  (102, 'WIRELESS MOUSE', 'ELECTRONICS', 25.00, 100),  
  (103, 'USB-C HUB', 'ELECTRONICS', 45.00, 60),  
  (104, 'MECHANICAL KEYBOARD', 'ELECTRONICS', 85.00, 40);
```

```
INSERT INTO stg_Orders  
  (OrderID, ProductID, CustomerName, Quantity, Discount, OrderDate)  
VALUES  
  (5001, 101, 'JOHN DOE', 2, 0.10, '2026-01-05'),  
  (5002, 102, 'ALICE SMITH', 5, 0.00, '2026-01-08'),  
  (5003, 103, 'BOB WHITE', 0, 0.00, '2026-01-10'),  
  (5004, 101, 'CHARLIE BROWN', 1, 0.00, '2026-02-02'),  
  (5005, 104, 'DAVID LEE', 3, 0.15, '2026-02-14');
```

```
-- =====  
-- STEP 4: CREATE TARGET TABLES  
-- =====
```

```
CREATE TABLE Fact_Sales (  
  SalesID INT AUTO_INCREMENT PRIMARY KEY,  
  OrderID INT,  
  ProductID INT,  
  ProductName VARCHAR(100),  
  Category VARCHAR(50),  
  CustomerName VARCHAR(100),  
  UnitPrice DECIMAL(10,2),  
  Quantity INT,  
  Discount DECIMAL(5,2),  
  GrossAmount DECIMAL(12,2),  
  DiscountAmount DECIMAL(12,2),  
  NetAmount DECIMAL(12,2),  
  DiscountCategory VARCHAR(20),  
  OrderDate DATE  
);
```

```
CREATE TABLE Rejected_Orders (  
  OrderID INT,  
  Reason VARCHAR(100)  
);
```

```
-- =====  
-- STEP 5: VALIDATE AND REJECT INVALID ORDERS  
-- =====
```

```
INSERT INTO Rejected_Orders  
  (OrderID, Reason)
```

```

SELECT
    o.OrderID,

    CASE
        WHEN p.ProductID IS NULL
            THEN 'Unknown product'

        WHEN o.Quantity IS NULL OR o.Quantity <= 0
            THEN 'Invalid quantity'

        WHEN o.Discount IS NULL
            OR o.Discount < 0
            OR o.Discount > 1
            THEN 'Invalid discount'

        ELSE 'Invalid record'
    END

FROM stg_Orders AS o

LEFT JOIN stg_Products AS p
    ON o.ProductID = p.ProductID

WHERE
    p.ProductID IS NULL
    OR o.Quantity IS NULL
    OR o.Quantity <= 0
    OR o.Discount IS NULL
    OR o.Discount < 0
    OR o.Discount > 1;

-- =====
-- STEP 6: TRANSFORM VALID DATA AND LOAD FACT TABLE
-- =====

INSERT INTO Fact_Sales
(
    OrderID,
    ProductID,
    ProductName,
    Category,
    CustomerName,
    UnitPrice,
    Quantity,
    Discount,
    GrossAmount,
    DiscountAmount,
    NetAmount,
    DiscountCategory,
    OrderDate
)

```

```

SELECT
  o.OrderID,
  o.ProductID,
  p.ProductName,
  p.Category,
  o.CustomerName,
  p.UnitPrice,
  o.Quantity,
  o.Discount,

  -- Gross Amount
  ROUND(
    p.UnitPrice * o.Quantity,
    2
  ),

  -- Discount Amount
  ROUND(
    p.UnitPrice * o.Quantity * o.Discount,
    2
  ),

  -- Net Amount
  ROUND(
    p.UnitPrice * o.Quantity * (1 - o.Discount),
    2
  ),

  -- Discount Category
  CASE
    WHEN o.Discount = 0
      THEN 'No Discount'

    WHEN o.Discount <= 0.10
      THEN 'Standard Discount'

    ELSE 'Premium Discount'
  END,

  o.OrderDate

FROM stg_Orders AS o

INNER JOIN stg_Products AS p
  ON o.ProductID = p.ProductID

WHERE
  o.Quantity > 0
  AND o.Discount BETWEEN 0 AND 1;

```

## Step 7: View Successfully Processed Sales (Fact\_Sales)

### Code

```
-- =====
-- STEP 7: VIEW SUCCESSFULLY PROCESSED SALES
-- =====
```

```
SELECT *
FROM Fact_Sales
```

Output

| SalesID | OrderID | ProductID | ProductName         | Category    | CustomerName  | UnitPrice | Quantity | Discount | GrossAmount | DiscountAmount | NetAmount | DiscountCategory  | OrderDate  |
|---------|---------|-----------|---------------------|-------------|---------------|-----------|----------|----------|-------------|----------------|-----------|-------------------|------------|
| 1       | 5001    | 101       | LAPTOP PRO 15       | ELECTRONICS | JOHN DOE      | 1200.00   | 2        | 0.10     | 2400.00     | 240.00         | 2160.00   | Standard Discount | 01/05/2026 |
| 2       | 5002    | 102       | WIRELESS MOUSE      | ELECTRONICS | ALICE SMITH   | 25.00     | 5        | 0.00     | 125.00      | 0.00           | 125.00    | No Discount       | 01/08/2026 |
| 3       | 5004    | 101       | LAPTOP PRO 15       | ELECTRONICS | CHARLIE BROWN | 1200.00   | 1        | 0.00     | 1200.00     | 0.00           | 1200.00   | No Discount       | 02/02/2026 |
| 4       | 5005    | 104       | MECHANICAL KEYBOARD | ELECTRONICS | DAVID LEE     | 85.00     | 3        | 0.15     | 255.00      | 38.25          | 216.75    | Premium Discount  | 02/14/2026 |

4 row(s) returned

Step 8: View Rejected Orders

Code

```
-- =====
-- STEP 8: VIEW REJECTED ORDERS
-- =====
```

```
SELECT *
FROM Rejected_Orders
```

Output

| OrderID | Reason           |
|---------|------------------|
| 5003    | Invalid quantity |

1 row(s) returned

Step 9: Category × Discount Category Report (ROLLUP)

Code

```
-- =====
-- STEP 9: CATEGORY × DISCOUNT CATEGORY REPORT
-- =====
```

```
SELECT
  COALESCE(Category, 'ALL CATEGORIES') AS Category,
```

```
COALESCE(
    DiscountCategory,
    'ALL DISCOUNTS'
) AS DiscountCategory,

COUNT(*) AS Orders,

SUM(Quantity) AS Units,

SUM(GrossAmount) AS Gross,

SUM(DiscountAmount) AS Discounts,

SUM(NetAmount) AS Net

FROM Fact_Sales

GROUP BY
    Category,
    DiscountCategory WITH ROLLUP;
```

Output

| Category       | DiscountCategory  | Orders | Units | Gross   | Discounts | Net     |
|----------------|-------------------|--------|-------|---------|-----------|---------|
| ALL CATEGORIES | ALL DISCOUNTS     | 4      | 11    | 3980.00 | 278.25    | 3701.75 |
| ELECTRONICS    | No Discount       | 2      | 6     | 1325.00 | 0.00      | 1325.00 |
| ELECTRONICS    | Premium Discount  | 1      | 3     | 255.00  | 38.25     | 216.75  |
| ELECTRONICS    | Standard Discount | 1      | 2     | 2400.00 | 240.00    | 2160.00 |
| ELECTRONICS    | ALL DISCOUNTS     | 4      | 11    | 3980.00 | 278.25    | 3701.75 |

5 row(s) returned

Step 10: Product-wise Sales Report

Code

```
-- =====
-- STEP 10: PRODUCT-WISE SALES REPORT
-- =====

SELECT
    ProductName,
    COUNT(*) AS Orders,
    SUM(Quantity) AS Units,
    SUM(NetAmount) AS Net

FROM Fact_Sales

GROUP BY ProductName

ORDER BY Net DESC;
```

Output

| ProductName         | Orders | Units | Net     |
|---------------------|--------|-------|---------|
| LAPTOP PRO 15       | 2      | 3     | 3360.00 |
| MECHANICAL KEYBOARD | 1      | 3     | 216.75  |
| WIRELESS MOUSE      | 1      | 5     | 125.00  |

3 row(s) returned

Step 11: Monthly Sales Report

Code

```
-- =====
-- STEP 11: MONTHLY SALES REPORT
-- =====

SELECT
    DATE_FORMAT(OrderDate, '%Y-%m') AS Month,

    COUNT(*) AS Orders,

    SUM(NetAmount) AS Net

FROM Fact_Sales

GROUP BY
    DATE_FORMAT(OrderDate, '%Y-%m')

ORDER BY Month;
```

Output

| Month   | Orders | Net     |
|---------|--------|---------|
| 2026-01 | 2      | 2285.00 |
| 2026-02 | 2      | 1416.75 |

2 row(s) returned

Step 12: Data Quality Check

Code

```
SELECT
    COUNT(*) AS Valid_Fact_Rows
FROM Fact_Sales;
```

Output

| Valid_Fact_Rows |
|-----------------|
| 4               |

*1 row(s) returned*

### Code

```
SELECT  
    COUNT(*) AS Rejected_Rows  
FROM Rejected_Orders;
```

### Output

| Rejected_Rows |
|---------------|
| 1             |

*1 row(s) returned*

Power BI Dashboard: Sales & Order Discounting Analytics

**Output (dashboard)**

# Sales & Order Discounting Analytics

ETL Data Model & Fact\_Sales Summary | Practical 2

\$3,980.00

Total Gross Sales

\$3,701.75

Total Net Revenue

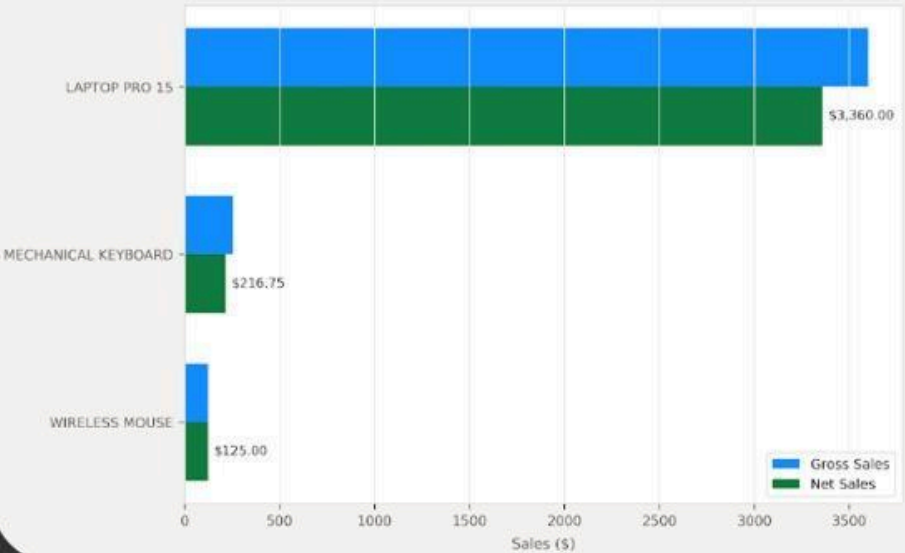
\$278.25

Total Discount Given

11

Total Quantity Sold

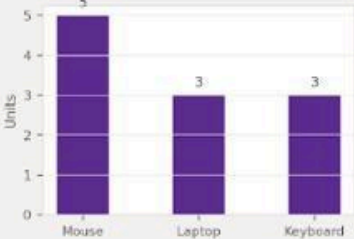
Revenue by Product (Gross vs Net)



Net Sales by Pricing Category



Units Sold per Item



Order Details & Calculations

| OrderID | Customer      | Product          | Qty | Price   | Disc | Total (\$) |
|---------|---------------|------------------|-----|---------|------|------------|
| 5001    | JOHN DOE      | LAPTOP PRO 15    | 2   | \$1,200 | 10%  | \$2,160.00 |
| 5002    | ALICE SMITH   | WIRELESS MOUS    | 5   | \$25    | 0%   | \$125.00   |
| 5004    | CHARLIE BROWN | LAPTOP PRO 15    | 1   | \$1,200 | 0%   | \$1,200.00 |
| 5005    | DAVID LEE     | MECHANICAL KEYBO | 3   | \$85    | 15%  | \$216.75   |

# Practical 3: Data Cleaning: Missing Values, Duplicates, and Normalization

## Aim

To write a Python program using Pandas to handle missing values, remove duplicate records, and normalize data using the Min-Max normalization technique.

## Code

```
import pandas as pd

# Sample Data
data = {
    'Name': ['Amit', 'Riya', 'Amit', 'Neha', None],
    'Age': [22, 24, 22, None, 26],
    'Marks': [85, 90, 85, 78, None]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Original Data:")
print(df)

print("\nMissing values per column:")
print(df.isnull().sum())

# -----
# 1. Handling Missing Values
# -----
print("\nHandling Missing Values...")

# Fill missing Age with mean
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Fill missing Marks with mean
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

# Fill missing Name with 'Unknown'
df['Name'] = df['Name'].fillna('Unknown')

print(df)

# -----
# 2. Removing Duplicate Rows
# -----
print("\nRemoving Duplicates...")
print("Duplicate rows found:", df.duplicated().sum())

df = df.drop_duplicates()

print(df)

# -----
# 3. Data Normalization (Min-Max)
# Formula: (x - min) / (max - min)
# -----
print("\nNormalizing Age and Marks...")

df['Age_Normalized'] = (df['Age'] - df['Age'].min()) / (df['Age'].max() - df['Age'].min())

df['Marks_Normalized'] = (df['Marks'] - df['Marks'].min()) / (df['Marks'].max() - df['Marks'].min())

print(df)

# Final Data
print("\nFinal Processed Data:")
print(df)
```

## Output

```
Original Data:
  Name  Age  Marks
0  Amit   22    85
1  Riya   24    90
2  Amit   22    85
3  Neha   None   78
4  None   26   None
```

```

0 Amit 22.0 85.0
1 Riya 24.0 90.0
2 Amit 22.0 85.0
3 Neha NaN 78.0
4 None 26.0 NaN
Missing values per column:
Name 1
Age 1
Marks 1
dtype: int64
Handling Missing Values...
  Name Age Marks
0 Amit 22.0 85.0
1 Riya 24.0 90.0
2 Amit 22.0 85.0
3 Neha 23.5 78.0
4 Unknown 26.0 84.5
Removing Duplicates...
Duplicate rows found: 1
  Name Age Marks
0 Amit 22.0 85.0
1 Riya 24.0 90.0
3 Neha 23.5 78.0
4 Unknown 26.0 84.5
Normalizing Age and Marks...
  Name Age Marks Age_Normalized Marks_Normalized
0 Amit 22.0 85.0 0.000 0.583333
1 Riya 24.0 90.0 0.500 1.000000
3 Neha 23.5 78.0 0.375 0.000000
4 Unknown 26.0 84.5 1.000 0.541667
Final Processed Data:
  Name Age Marks Age_Normalized Marks_Normalized
0 Amit 22.0 85.0 0.000 0.583333
1 Riya 24.0 90.0 0.500 1.000000
3 Neha 23.5 78.0 0.375 0.000000
4 Unknown 26.0 84.5 1.000 0.541667

```

# Practical 4: Noise Elimination, Feature Selection, and Exploratory Data Analysis (EDA)

## Aim

To write a Python program demonstrating noise elimination (outlier removal), feature selection, and exploratory data analysis using Pandas, NumPy, Matplotlib, and Scikit-learn.

## Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# -----
# Sample Dataset
# -----
data = {
    'Age': [20, 21, 22, 23, 24, 100],      # 100 is an outlier (noise)
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]      # Constant feature
}
df = pd.DataFrame(data)

print("Original Dataset:")
print(df)
```

## Output

```
Original Dataset:
   Age  Marks  Attendance  Constant
0   20     75         85         1
1   21     80         88         1
2   22     85         90         1
3   23     90         92         1
4   24     95         94         1
5  100     20         50         1
```

## Code

```
# =====
# 1. Noise Elimination (IQR Outlier Removal)
# =====
mask = pd.Series(True, index=df.index)

for col in df.select_dtypes(include=np.number).columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    print(f"{col}: lower={lower:.2f}, upper={upper:.2f}")
    mask &= df[col].between(lower, upper)

df_clean = df[mask]

print("\nDataset after Noise Elimination:")
print(df_clean)
```

## Output

```
Age: lower=17.50, upper=27.50
Marks: lower=57.50, upper=107.50
Attendance: lower=77.12, upper=100.12
Constant: lower=1.00, upper=1.00
Dataset after Noise Elimination:
   Age  Marks  Attendance  Constant
0   20     75         85         1
1   21     80         88         1
2   22     85         90         1
3   23     90         92         1
```

## Code

```
# =====
# 2. Feature Selection (remove constant features)
# =====
selector = VarianceThreshold(threshold=0.0)
selector.fit(df_clean)
selected_columns = df_clean.columns[selector.get_support()]

print("\nSelected Features:")
print(list(selected_columns))

df_selected = df_clean[selected_columns]
```

## Output

```
Selected Features:
['Age', 'Marks', 'Attendance']
```

## Code

```
# =====
# 3. Exploratory Data Analysis (EDA)
# =====
print("\nDataset Information:")
df_selected.info()

print("\nStatistical Summary:")
print(df_selected.describe())

print("\nCorrelation Matrix:")
print(df_selected.corr())
```

## Output

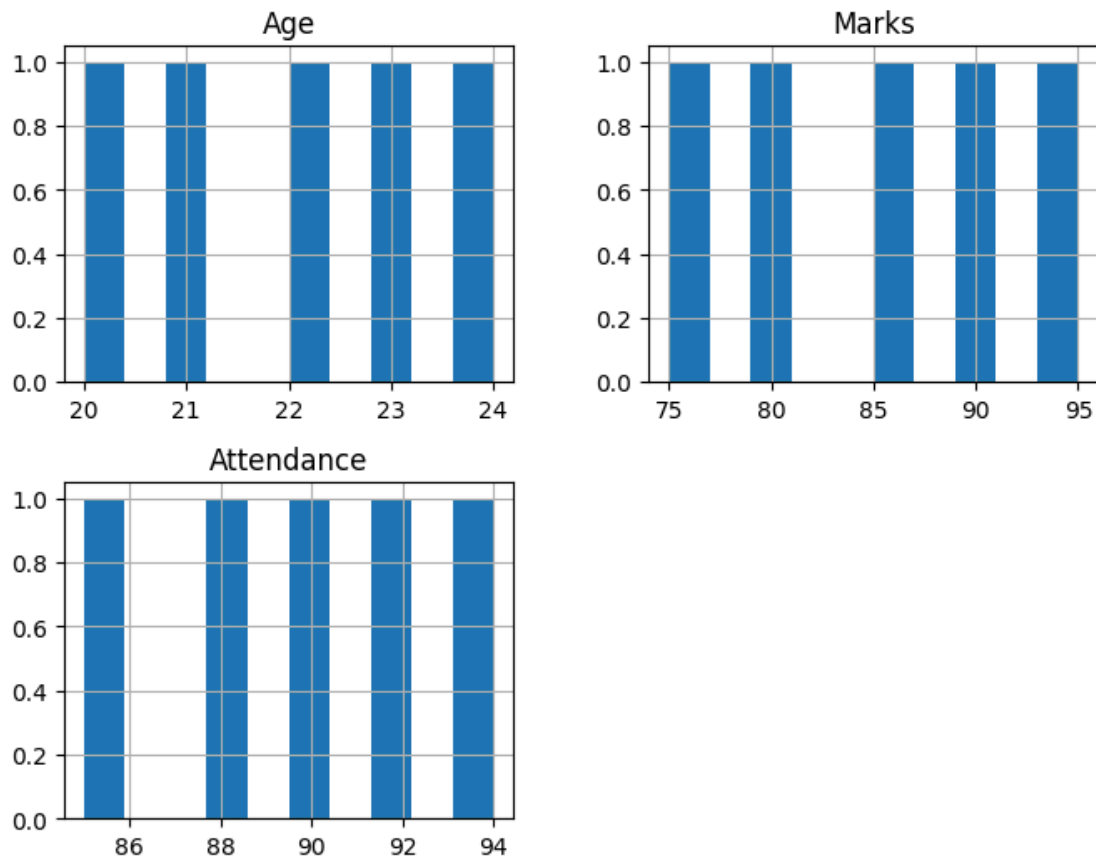
```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column    Non-Null Count  Dtype
---  -
0    Age         5 non-null     int64
1    Marks       5 non-null     int64
2    Attendance  5 non-null     int64
dtypes: int64(3)
memory usage: 160.0 bytes
Statistical Summary:
      Age  Marks  Attendance
count  5.000000  5.000000   5.000000
mean   22.000000  85.000000  89.800000
std    1.581139   7.905694   3.49285
min    20.000000  75.000000  85.000000
25%    21.000000  80.000000  88.000000
50%    22.000000  85.000000  90.000000
75%    23.000000  90.000000  92.000000
max    24.000000  95.000000  94.000000
Correlation Matrix:
      Age  Marks  Attendance
Age      1.000000  1.000000   0.995893
Marks    1.000000  1.000000   0.995893
Attendance 0.995893  0.995893   1.000000
```

## Code

```
# Histogram
df_selected.hist(figsize=(8, 6))
plt.suptitle("Histogram of Features")
plt.show()
```

## Output (chart)

## Histogram of Features

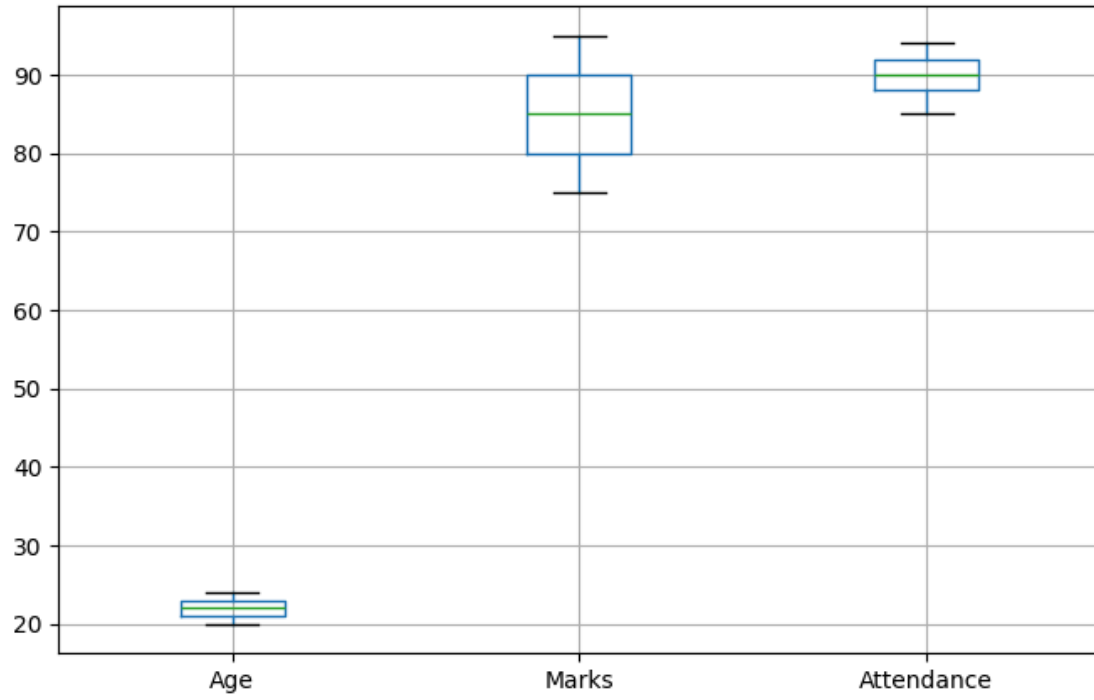


### Code

```
# Boxplot
df_selected.boxplot(figsize=(8, 5))
plt.title("Boxplot of Dataset")
plt.show()
```

### Output (chart)

Boxplot of Dataset



## Practical 5: Extracting Data from APIs and Flat Files

### Aim

To write a Python script that fetches user data from a public REST API, cleans it, and merges it with a flat CSV file containing location details.

### Code

```
import pandas as pd
import requests

def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        # Normalizing nested JSON into a flat table
        return pd.json_normalize(data)
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()

def extract_csv_data(file_path):
    """Reads a flat CSV file."""
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()
```

### Code

```
# 1. API Extraction
api_url = "https://jsonplaceholder.typicode.com/users"
api_df = extract_api_data(api_url)

if not api_df.empty:
    print("Raw API columns:")
    print(list(api_df.columns))

    # Cleaning: keep only required columns and rename
    api_df = api_df[["id", "name", "email", "company.name"]]
    api_df = api_df.rename(columns={"company.name": "company"})

    print("\nCleaned API data:")
    print(api_df.head())
```

### Output

```
Raw API columns:
['id', 'name', 'username', 'email', 'phone', 'website', 'address.street', 'address.suite', 'address.city', 'address.zipcode', 'address.geo.lat', 'address.geo.lng', 'company.name', 'company.catchPhrase', 'company.bs']
Cleaned API data:
   id  name          email          company
0  1  Leanne Graham  Sincere@april.biz  Romaguera-Crona
1  2  Ervin Howell   Shanna@melissa.tv  Deckow-Crist
2  3  Clementine Bauch  Nathan@yesenia.net  Romaguera-Jacobson
3  4  Patricia Lebsack  Julianne.OConner@kory.org  Robel-Corkery
4  5  Chelsey Dietrich  Lucio_Hettinger@annie.ca  Keebler LLC
```

### Code

```
# 2. Flat File Extraction (create a mock CSV first)
csv_file = "locations.csv"

pd.DataFrame(
    {
        "id": list(range(1, 11)),
        "city": ["New York", "London", "Paris", "Tokyo", "Berlin",
                 "Delhi", "Sydney", "Moscow", "Cairo", "Beijing"],
    }
)
```

```

        "country": ["USA", "UK", "France", "Japan", "Germany",
                    "India", "Australia", "Russia", "Egypt", "China"],
    }
).to_csv(csv_file, index=False)

csv_df = extract_csv_data(csv_file)
print("\nCSV data:")
print(csv_df.head())

```

## Output

```

CSV data:
  id  city country
0  1  New York   USA
1  2   London    UK
2  3    Paris  France
3  4   Tokyo   Japan
4  5   Berlin  Germany

```

## Code

```

# 3. Transformation & Merging
if not api_df.empty and not csv_df.empty:
    merged_df = pd.merge(api_df, csv_df, on="id", how="inner")

    print("\n--- Merged ETL Pipeline Data View ---")
    print(merged_df)

    merged_df.to_csv("cleaned_warehouse_profiles.csv", index=False)
    print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")
else:
    print("\nMerge skipped: one of the sources is empty.")

```

## Output

```

--- Merged ETL Pipeline Data View ---
  id      name      email \
0  1  Leanne Graham  Sincere@april.biz
1  2   Ervin Howell  Shanna@melissa.tv
2  3  Clementine Bauch  Nathan@yesenia.net
3  4  Patricia Lebsack  Julianne.OConner@kory.org
4  5   Chelsey Dietrich  Lucio_Hettinger@annie.ca
5  6  Mrs. Dennis Schulist  Karley_Dach@jasper.info
6  7   Kurtis Weissnat  Telly.Hoeger@billy.biz
7  8  Nicholas Runolfsdottir V  Sherwood@rosamond.me
8  9   Glenna Reichert  Chaim_McDermott@dana.io
9  10  Clementina DuBuque  Rey.Padberg@karina.biz
   company  city  country
0  Romaguera-Crona  New York   USA
1   Deckow-Crist  London    UK
2  Romaguera-Jacobson  Paris  France
3   Robel-Corkery  Tokyo   Japan
4   Keebler LLC  Berlin  Germany
5  Considine-Lockman  Delhi  India
6   Johns Group  Sydney  Australia
7  Abernathy Group  Moscow  Russia
8   Yost and Sons  Cairo  Egypt
9   Hoeger LLC  Beijing  China
Data successfully saved to 'cleaned_warehouse_profiles.csv'

```

## Practical 6: DAG Orchestration using Apache Airflow

### Aim

To configure a Directed Acyclic Graph (DAG) in Apache Airflow that runs daily, executing sequential dependent tasks: a start notifier, an extraction script, and a pipeline success logger.

### Install Airflow 3

#### Code

```
import sys
PY = f'{sys.version_info.major}.{sys.version_info.minor}'
AF = "3.1.0" if PY == "3.13" else "3.0.6"
print("Python", PY, "-> installing Airflow", AF)
!pip install -q "apache-airflow=={AF}" --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-{AF}/constraints-{PY}.txt"
```

#### Output

```
Python 3.13 -> installing Airflow 3.1.0
```

#### Code

```
!airflow version
```

#### Output

```
3.1.0
```

### Environment, Extraction script and DAG file

#### Code

```
import os
os.environ["AIRFLOW_HOME"] = "/content/airflow"
os.environ["AIRFLOW__CORE__LOAD_EXAMPLES"] = "False"
os.makedirs("/content/airflow/dags", exist_ok=True)

# ----- data_extraction.py (Practical 5) -----
extraction = """# Name: Aditya Dhage
import pandas as pd
import requests

def extract_api_data(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return pd.json_normalize(response.json())
    except requests.exceptions.RequestException as e:
        print(f'API Error: {e}')
        return pd.DataFrame()

def extract_csv_data(file_path):
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f'File Error: {e}')
        return pd.DataFrame()

if __name__ == "__main__":
    print("Name: Aditya Dhage")
    api_df = extract_api_data("https://jsonplaceholder.typicode.com/users")
    if not api_df.empty:
        api_df = api_df[["id", "name", "email", "company.name"]]
        api_df.rename(columns={"company.name": "company"}, inplace=True)

    csv_file = "/content/locations.csv"
    pd.DataFrame({
        "id": list(range(1, 11)),
        "city": ["New York", "London", "Paris", "Tokyo", "Berlin",
```

```

        "Delhi", "Sydney", "Moscow", "Cairo", "Beijing"],
        "country": ["USA", "UK", "France", "Japan", "Germany",
                    "India", "Australia", "Russia", "Egypt", "China"],
    }).to_csv(csv_file, index=False)
    csv_df = extract_csv_data(csv_file)

    if not api_df.empty and not csv_df.empty:
        merged_df = pd.merge(api_df, csv_df, on="id", how="inner")
        print("\n--- Merged ETL Pipeline Data View ---")
        print(merged_df)
        merged_df.to_csv("/content/cleaned_warehouse_profiles.csv", index=False)
        print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")
    else:
        print("\nMerge skipped: one of the sources is empty.")
'''
open("/content/data_extraction.py", "w").write(extraction)

# ----- university_etl_orchestration.py (Practical 6) -----
dag_code = '''# Name: Aditya Dhage
from datetime import datetime, timedelta

from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

# Default arguments for the workflow pipeline execution
default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="Lab assignment for API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    # Start task
    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )

    # Execute extraction script
    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /content/data_extraction.py",
    )

    # End task
    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command="echo \"ETL Execution completed successfully at $(date)\"",
    )

    # Define workflow
    start_pipeline >> execute_extraction >> pipeline_complete
'''
open("/content/airflow/dags/university_etl_orchestration.py", "w").write(dag_code)

print("Written:", os.listdir("/content/airflow/dags"), "and /content/data_extraction.py")

```

## Output

```
Written: ['university_etl_orchestration.py'] and /content/data_extraction.py
```

## Initialise Airflow and check the DAG

## Code

```
!airflow db migrate 2>&1 | tail -3
print("\n--- DAG registered ---")
!airflow dags list 2>&1 | grep university
print("\n--- Import errors (should be empty) ---")
!airflow dags list-import-errors 2>&1 | tail -5
print("\n--- Task order ---")
!airflow tasks list university_etl_orchestration --tree 2>&1 | tail -8
```

## Output

```
DB: sqlite:///content/airflow/airflow.db
Performing upgrade to the metadata database sqlite:///content/airflow/airflow.db
Database migrating done!
--- DAG registered ---
--- Import errors (should be empty) ---
No data found
--- Task order ---
    Rotate encrypted connection credentials and variables
scheduler      Start a scheduler instance
standalone      Run an all-in-one copy of Airflow
triggerer       Start a triggerer instance
version         Show the version
Options:
-h, --help      show this help message and exit
```

## Run the DAG

## Code

```
!airflow dags test university_etl_orchestration 2026-01-01 2>&1 | grep -E "DAG TEST|Running command|Name:|Merged|successfully|return code|DagRun Finished"
```

## Output

```
2026-09-28T14:07:47.669094Z [info ] [DAG TEST] starting task_id=start_pipeline map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1326
2026-09-28T14:07:47.669852Z [info ] [DAG TEST] running task <TaskInstance: university_etl_orchestration.start_pipeline
manual_2026-09-28T14:07:47.576434+00:00 [TaskInstanceState.SCHEDULED]> [airflow.sdk.definitions.dag] loc=dag.py:1329
2026-09-28T14:07:50.537372Z [info ] [DAG TEST] end task task_id=start_pipeline map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1383
2026-09-28T14:07:50.552903Z [info ] [DAG TEST] starting task_id=run_extraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1326
2026-09-28T14:07:50.553207Z [info ] [DAG TEST] running task <TaskInstance: university_etl_orchestration.run_extraction_script
manual_2026-09-28T14:07:47.576434+00:00 [TaskInstanceState.SCHEDULED]> [airflow.sdk.definitions.dag] loc=dag.py:1329
2026-09-28T14:07:52.777963Z [info ] Running command: ['/usr/bin/bash', '-c', 'python3 /content/data_extraction.py']
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:88
2026-09-28T14:07:53.660519Z [info ] Name: Aditya Dhage [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
loc=subprocess.py:106
2026-09-28T14:07:53.660894Z [info ] --- Merged ETL Pipeline Data View ---
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:106
2026-09-28T14:07:53.662213Z [info ] Data successfully saved to 'cleaned_warehouse_profiles.csv'
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:106
2026-09-28T14:07:53.823403Z [info ] Command exited with return code 0
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:110
2026-09-28T14:07:53.882221Z [info ] [DAG TEST] end task task_id=run_extraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1383
2026-09-28T14:07:53.896221Z [info ] [DAG TEST] starting task_id=log_pipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1326
2026-09-28T14:07:53.896538Z [info ] [DAG TEST] running task <TaskInstance: university_etl_orchestration.log_pipeline_success
manual_2026-09-28T14:07:47.576434+00:00 [TaskInstanceState.SCHEDULED]> [airflow.sdk.definitions.dag] loc=dag.py:1329
2026-09-28T14:07:55.871604Z [info ] Running command: ['/usr/bin/bash', '-c', 'echo "ETL Execution completed successfully at $(date)"]
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:88
2026-09-28T14:07:55.891082Z [info ] ETL Execution completed successfully at Mon Sep 28 02:07:55 PM UTC 2026
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:106
2026-09-28T14:07:55.891405Z [info ] Command exited with return code 0
[airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:110
2026-09-28T14:07:55.949958Z [info ] [DAG TEST] end task task_id=log_pipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1383
2026-09-28T14:07:55.953725Z [info ] DagRun Finished: dag_id=university_etl_orchestration, logical_date=2026-01-01 00:00:00+00:00,
run_id>manual_2026-09-28T14:07:47.576434+00:00, run_start_date=2026-01-01 00:00:00+00:00, run_end_date=2026-09-28 14:07:55.953589+00:00,
run_duration=23378875.953589, state=success, run_type>manual, data_interval_start=2026-01-01 00:00:00+00:00, data_interval_end=2026-01-01
00:00:00+00:00, [airflow.models.dagrun.DagRun] loc=dagrun.py:1274
```

# Practical 7: ETL Pipeline Design and Implementation

## Aim

To design ETL pipelines that extract data from single and multiple CSV files and JSON sources, clean and transform it, identify and remove invalid records, validate data before loading, and implement incremental (watermark-based) loading into a database.

## Code

```
import pandas as pd
import numpy as np
import sqlite3
import json
import os
import glob

# =====
# SETUP: sample data + SQLite database
# =====

os.makedirs("data", exist_ok=True)
DB = "etl_practical7.db"
if os.path.exists(DB):
    os.remove(DB)
conn = sqlite3.connect(DB)

cols = ["order_id", "customer", "product", "quantity", "price", "order_date"]

jan = pd.DataFrame([
    [1, "alice", "Laptop", 1, 55000, "2026-01-05"],
    [2, "BOB", "Mouse", 2, 500, "2026-01-10"],
    [3, "Charlie", "Keyboard", 1, 1500, "2026-01-15"],
    [3, "Charlie", "Keyboard", 1, 1500, "2026-01-15"], # duplicate
    [4, None, "Monitor", 1, 12000, "2026-01-20"], # missing customer
    [5, "David", "Mouse", -2, 500, "2026-01-22"], # negative quantity
    [6, "Eva", "Laptop", 1, 52000, "2026-01-25"],
], columns=cols)

feb = pd.DataFrame([
    [7, "Frank", "Monitor", 2, 11000, "2026-02-03"],
    [8, "Grace", "Keyboard", 1, 1400, "not-a-date"], # invalid date
    [9, "Heidi", "Mouse", 3, -450, "2026-02-10"], # negative price
    [10, "Ivan", "Laptop", 1, 54000, "2026-02-18"],
], columns=cols)

mar = pd.DataFrame([
    [2, "Bob", "Mouse", 2, 500, "2026-01-10"], # old record (already loaded)
    [10, "Ivan", "Laptop", 1, 54000, "2026-02-18"], # already loaded
    [11, "Judy", "Mouse", 2, 520, "2026-03-02"], # new
    [12, "Karl", "Monitor", 1, 12500, "2026-03-09"], # new
    [13, "Liam", "Mouse", 0, 500, "2026-03-15"], # invalid quantity
], columns=cols)

jan.to_csv("data/sales_jan.csv", index=False)
feb.to_csv("data/sales_feb.csv", index=False)
mar.to_csv("data/sales_mar.csv", index=False)

users_json = [
    {"id": 1, "name": "Asha Rao", "email": "ASHA@Example.com",
     "address": {"city": "Mumbai", "zipcode": "400001"}, "company": {"name": "TechCorp"}},
    {"id": 2, "name": "Rohan Das", "email": "rohan@example.com",
     "address": {"city": "Pune", "zipcode": "411001"}, "company": {"name": "DataWorks"}},
    {"id": 3, "name": "Meera Iyer", "email": "meera@example.com",
     "address": {"city": "Chennai", "zipcode": "600001"}, "company": {"name": "CloudSoft"}}
]

with open("data/users.json", "w") as f:
    json.dump(users_json, f, indent=2)
```

## Code

```
# =====
# REUSABLE FUNCTIONS
# =====
```

```
def clean(df):
    """Cleaning: trim text, fix case, fix types, drop exact duplicates."""
    df = df.copy()
    for c in ["customer", "product"]:
        df[c] = df[c].str.strip().str.title()
        df[c] = df[c].replace("", np.nan)
    df["order_id"] = pd.to_numeric(df["order_id"], errors="coerce")
    df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
    df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce").dt.strftime("%Y-%m-%d")
    return df.drop_duplicates().reset_index(drop=True)

def transform(df):
    """Transformation: derive total_amount."""
    df = df.copy()
    df["total_amount"] = df["quantity"] * df["price"]
    return df

def validate(df):
    """Validation rules. Returns (valid_df, invalid_df_with_reason)."""
    reasons = pd.Series("", index=df.index)
    rules = {
        "missing order_id": df["order_id"].isna(),
        "missing customer": df["customer"].isna(),
        "invalid quantity": df["quantity"].isna() | (df["quantity"] <= 0),
        "invalid price": df["price"].isna() | (df["price"] <= 0),
        "invalid date": df["order_date"].isna(),
    }
    for msg, cond in rules.items():
        reasons.loc[cond] = reasons.loc[cond] + msg + "; "
    bad = reasons != ""
    return df[~bad].copy(), df[bad].assign(reason=reasons[bad])
```

## Task 1: ETL from a single CSV -> clean -> transform -> database

### Code

```
print("=" * 60)
print("TASK 1: Single CSV ETL")
print("=" * 60)
raw = pd.read_csv("data/sales_jan.csv")          # Extract
print("Raw data:\n", raw, "\n")

t1 = transform(clean(raw))                        # Clean + Transform
t1_valid, _ = validate(t1)
t1_valid.to_sql("sales_jan_clean", conn, if_exists="replace", index=False) # Load

print("Loaded into table 'sales_jan_clean':")
print(pd.read_sql("SELECT * FROM sales_jan_clean", conn))
```

### Output

```
=====
TASK 1: Single CSV ETL
=====
```

```
Raw data:
```

|   | order_id | customer | product  | quantity | price | order_date |
|---|----------|----------|----------|----------|-------|------------|
| 0 | 1        | alice    | Laptop   | 1        | 55000 | 2026-01-05 |
| 1 | 2        | BOB      | Mouse    | 2        | 500   | 2026-01-10 |
| 2 | 3        | Charlie  | Keyboard | 1        | 1500  | 2026-01-15 |
| 3 | 3        | Charlie  | Keyboard | 1        | 1500  | 2026-01-15 |
| 4 | 4        | NaN      | Monitor  | 1        | 12000 | 2026-01-20 |
| 5 | 5        | David    | Mouse    | -2       | 500   | 2026-01-22 |
| 6 | 6        | Eva      | Laptop   | 1        | 52000 | 2026-01-25 |

```
Loaded into table 'sales_jan_clean':
```

|   | order_id | customer | product  | quantity | price | order_date | total_amount |
|---|----------|----------|----------|----------|-------|------------|--------------|
| 0 | 1        | Alice    | Laptop   | 1        | 55000 | 2026-01-05 | 55000        |
| 1 | 2        | Bob      | Mouse    | 2        | 500   | 2026-01-10 | 1000         |
| 2 | 3        | Charlie  | Keyboard | 1        | 1500  | 2026-01-15 | 1500         |
| 3 | 6        | Eva      | Laptop   | 1        | 52000 | 2026-01-25 | 52000        |

## TASK 2: Extract multiple CSV files and combine

### Code

```
print("\n" + "=" * 60)
print("TASK 2: Combine multiple CSV files")
print("=" * 60)
files = sorted(glob.glob("data/sales_*.csv"))
print("Files found:", files)

combined = pd.concat(
    [pd.read_csv(f).assign(source_file=os.path.basename(f)) for f in files],
    ignore_index=True,
)
print("Combined shape:", combined.shape)
print(combined.groupby("source_file").size())
```

### Output

```
=====
TASK 2: Combine multiple CSV files
=====
Files found: ['data/sales_feb.csv', 'data/sales_jan.csv', 'data/sales_mar.csv']
Combined shape: (16, 7)
source_file
sales_feb.csv    4
sales_jan.csv    7
sales_mar.csv    5
dtype: int64
```

## TASK 3: JSON -> transform selected fields -> relational DB

### Code

```
print("\n" + "=" * 60)
print("TASK 3: JSON to relational database")
print("=" * 60)
with open("data/users.json") as f:
    users_raw = json.load(f)

users = pd.json_normalize(users_raw)          # flatten nested JSON
users = users[["id", "name", "email", "address.city", "company.name"]]
users = users.rename(columns={"id": "user_id", "address.city": "city",
                              "company.name": "company"})
users["email"] = users["email"].str.lower()   # transform
users["email_domain"] = users["email"].str.split("@").str[1] # derived field
users.to_sql("users", conn, if_exists="replace", index=False)

print(pd.read_sql("SELECT * FROM users", conn))
```

### Output

```
=====
TASK 3: JSON to relational database
=====
 user_id  name      email      city  company email_domain
0      1  Asha Rao  asha@example.com  Mumbai  TechCorp  example.com
1      2  Rohan Das  rohan@example.com   Pune  DataWorks  example.com
2      3  Meera Iyer  meera@example.com  Chennai  CloudSoft  example.com
```

## TASK 4: Identify and remove invalid records

### Code

```
print("\n" + "=" * 60)
print("TASK 4: Identify and remove invalid records")
print("=" * 60)
all_clean = transform(clean(combined))
valid_all, invalid_all = validate(all_clean)
print(f"Total after cleaning: {len(all_clean)} | Valid: {len(valid_all)} | Invalid: {len(invalid_all)}")
print("\nInvalid records with reasons:")
print(invalid_all[["order_id", "customer", "quantity", "price", "order_date", "reason"]])
```

### Output

#### TASK 4: Identify and remove invalid records

Total after cleaning: 15 | Valid: 10 | Invalid: 5

Invalid records with reasons:

|    | order_id | customer | quantity | price | order_date | reason            |
|----|----------|----------|----------|-------|------------|-------------------|
| 1  | 8        | Grace    | 1        | 1400  | NaN        | invalid date;     |
| 2  | 9        | Heidi    | 3        | -450  | 2026-02-10 | invalid price;    |
| 7  | 4        | NaN      | 1        | 12000 | 2026-01-20 | missing customer; |
| 8  | 5        | David    | -2       | 500   | 2026-01-22 | invalid quantity; |
| 14 | 13       | Liam     | 0        | 500   | 2026-03-15 | invalid quantity; |

## TASKS 5 + 6: Validation before load + Incremental loading

### Code

```
conn.execute("""CREATE TABLE IF NOT EXISTS sales_clean (
    order_id INTEGER PRIMARY KEY, customer TEXT, product TEXT,
    quantity INTEGER, price REAL, order_date TEXT,
    total_amount REAL, source_file TEXT)""")
conn.execute("""CREATE TABLE IF NOT EXISTS rejected_records (
    order_id REAL, customer TEXT, product TEXT, quantity REAL, price REAL,
    order_date TEXT, total_amount REAL, source_file TEXT, reason TEXT)""")
conn.execute("""CREATE TABLE IF NOT EXISTS etl_watermark (
    pipeline TEXT PRIMARY KEY, last_order_date TEXT, last_run TEXT)""")
conn.commit()

REQUIRED = set(cols)

def get_watermark():
    row = conn.execute(
        "SELECT last_order_date FROM etl_watermark WHERE pipeline='sales'").fetchone()
    return row[0] if row else None

def run_pipeline(file_list, run_name):
    print(f"\n---- {run_name} ----")
    frames = []
    for f in file_list:
        df = pd.read_csv(f)
        # TASK 5 (gate 1): schema validation before anything is loaded
        missing = REQUIRED - set(df.columns)
        if missing:
            print(f"ABORT: {f} is missing columns {missing}")
            return
        frames.append(df.assign(source_file=os.path.basename(f)))

    df = transform(clean(pd.concat(frames, ignore_index=True)))

    # TASK 5 (gate 2): row-level validation before loading
    valid, invalid = validate(df)
    if len(invalid):
        invalid.to_sql("rejected_records", conn, if_exists="append", index=False)

    # TASK 6: incremental filter using the watermark
    wm = get_watermark()
    print("Watermark before run:", wm)
    if wm:
        new = valid[valid["order_date"] >= wm]  # >= plus INSERT OR IGNORE = safe
    else:
        new = valid
    skipped_old = len(valid) - len(new)

    rows = new[["order_id", "customer", "product", "quantity", "price",
        "order_date", "total_amount", "source_file"]]
    rows = rows.astype({"order_id": int, "quantity": int}).astype(object).values.tolist()

    before = conn.total_changes
    conn.executemany("INSERT OR IGNORE INTO sales_clean VALUES (?, ?, ?, ?, ?, ?)", rows)
    inserted = conn.total_changes - before

    if len(valid):
        new_wm = valid["order_date"].max()
```

```

if wm:
    new_wm = max(new_wm, wm)
    conn.execute("INSERT OR REPLACE INTO etl_watermark VALUES ('sales', ?, datetime('now'))",
        (new_wm,))
    conn.commit()

print(f"Read: {len(df)} | Rejected: {len(invalid)} | Skipped by watermark: {skipped_old} "
    f"| Already in DB (ignored): {len(new) - inserted} | Newly inserted: {inserted}")

print("\n" + "=" * 60)
print("TASKS 5 & 6: Validation before load + Incremental loading")
print("=" * 60)

# Run 1: initial (historical) load
run_pipeline(["data/sales_jan.csv", "data/sales_feb.csv"], "RUN 1: initial load (Jan + Feb)")
print(pd.read_sql("SELECT * FROM sales_clean", conn))

# Run 2: new file arrives, only new records should be added
run_pipeline(["data/sales_mar.csv"], "RUN 2: incremental load (Mar)")

print("\nFinal table 'sales_clean':")
print(pd.read_sql("SELECT * FROM sales_clean ORDER BY order_id", conn))

print("\nRejected records log:")
print(pd.read_sql("SELECT order_id, customer, quantity, price, order_date, reason FROM rejected_records", conn))

print("\nWatermark table:")
print(pd.read_sql("SELECT * FROM etl_watermark", conn))

print("\nTables in database:")
print(pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", conn))

conn.close()

```

## Output

```
=====
TASKS 5 & 6: Validation before load + Incremental loading
=====
```

```
----- RUN 1: initial load (Jan + Feb) -----
```

```
Watermark before run: None
```

```
Read: 10 | Rejected: 4 | Skipped by watermark: 0 | Already in DB (ignored): 0 | Newly inserted: 6
```

|   | order_id | customer | product  | quantity | price   | order_date | total_amount \ |
|---|----------|----------|----------|----------|---------|------------|----------------|
| 0 | 1        | Alice    | Laptop   | 1        | 55000.0 | 2026-01-05 | 55000.0        |
| 1 | 2        | Bob      | Mouse    | 2        | 500.0   | 2026-01-10 | 1000.0         |
| 2 | 3        | Charlie  | Keyboard | 1        | 1500.0  | 2026-01-15 | 1500.0         |
| 3 | 6        | Eva      | Laptop   | 1        | 52000.0 | 2026-01-25 | 52000.0        |
| 4 | 7        | Frank    | Monitor  | 2        | 11000.0 | 2026-02-03 | 22000.0        |
| 5 | 10       | Ivan     | Laptop   | 1        | 54000.0 | 2026-02-18 | 54000.0        |

```
source_file
```

```

0 sales_jan.csv
1 sales_jan.csv
2 sales_jan.csv
3 sales_jan.csv
4 sales_feb.csv
5 sales_feb.csv

```

```
----- RUN 2: incremental load (Mar) -----
```

```
Watermark before run: 2026-02-18
```

```
Read: 5 | Rejected: 1 | Skipped by watermark: 1 | Already in DB (ignored): 1 | Newly inserted: 2
```

```
Final table 'sales_clean':
```

|   | order_id | customer | product  | quantity | price   | order_date | total_amount \ |
|---|----------|----------|----------|----------|---------|------------|----------------|
| 0 | 1        | Alice    | Laptop   | 1        | 55000.0 | 2026-01-05 | 55000.0        |
| 1 | 2        | Bob      | Mouse    | 2        | 500.0   | 2026-01-10 | 1000.0         |
| 2 | 3        | Charlie  | Keyboard | 1        | 1500.0  | 2026-01-15 | 1500.0         |
| 3 | 6        | Eva      | Laptop   | 1        | 52000.0 | 2026-01-25 | 52000.0        |
| 4 | 7        | Frank    | Monitor  | 2        | 11000.0 | 2026-02-03 | 22000.0        |
| 5 | 10       | Ivan     | Laptop   | 1        | 54000.0 | 2026-02-18 | 54000.0        |
| 6 | 11       | Judy     | Mouse    | 2        | 520.0   | 2026-03-02 | 1040.0         |
| 7 | 12       | Karl     | Monitor  | 1        | 12500.0 | 2026-03-09 | 12500.0        |

```
source_file
```

```

0 sales_jan.csv
1 sales_jan.csv
2 sales_jan.csv

```

```
3 sales_jan.csv
4 sales_feb.csv
5 sales_feb.csv
6 sales_mar.csv
7 sales_mar.csv
Rejected records log:
  order_id customer quantity price order_date      reason
0    4.0   None    1.0 12000.0 2026-01-20 missing customer;
1    5.0   David   -2.0   500.0 2026-01-22 invalid quantity;
2    8.0   Grace    1.0  1400.0      None invalid date;
3    9.0   Heidi    3.0  -450.0 2026-02-10 invalid price;
4   13.0    Liam    0.0   500.0 2026-03-15 invalid quantity;
Watermark table:
  pipeline last_order_date      last_run
0  sales    2026-03-09 2026-09-28 14:08:25
Tables in database:
      name
0 sales_jan_clean
1      users
2   sales_clean
3 rejected_records
4    etl_watermark
```

## Practical 8: Big Data Processing with PySpark

### Aim

To use PySpark to read a CSV dataset and display its schema, perform filtering, grouping and aggregation, remove duplicate records, join two datasets, and calculate average sales by product category.

### Code

```
!pip install -q pyspark

import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

# =====
# SETUP: Spark session + sample CSV files
# =====
spark = SparkSession.builder.appName("Practical8_PySpark").master("local[*]").getOrCreate()

sales_rows = [
    (1, 101, 2, 500.0, "North", "2026-01-05"),
    (2, 102, 1, 1500.0, "South", "2026-01-06"),
    (3, 103, 3, 200.0, "East", "2026-01-07"),
    (4, 101, 1, 500.0, "West", "2026-01-08"),
    (5, 104, 2, 12000.0, "North", "2026-01-09"),
    (5, 104, 2, 12000.0, "North", "2026-01-09"), # duplicate
    (6, 105, 5, 80.0, "South", "2026-01-10"),
    (7, 102, 2, 1500.0, "East", "2026-01-11"),
    (8, 106, 1, 55000.0, "West", "2026-01-12"),
    (8, 106, 1, 55000.0, "West", "2026-01-12"), # duplicate
    (9, 103, 4, 200.0, "North", "2026-01-13"),
    (10, 107, 1, 3000.0, "South", "2026-01-14"), # product 107 not in products.csv
]
pd.DataFrame(sales_rows, columns=[
    "sale_id", "product_id", "quantity", "unit_price", "region", "sale_date"
]).to_csv("sales.csv", index=False)

products_rows = [
    (101, "Mouse", "Accessories"),
    (102, "Keyboard", "Accessories"),
    (103, "Pen Drive", "Accessories"),
    (104, "Monitor", "Electronics"),
    (105, "Notebook", "Stationery"),
    (106, "Laptop", "Electronics"),
]
pd.DataFrame(products_rows, columns=[
    "product_id", "product_name", "category"
]).to_csv("products.csv", index=False)
```

### TASK 1: Read a CSV dataset and display its schema

#### Code

```
print("=" * 60)
print("TASK 1: Read CSV and display schema")
print("=" * 60)

sales_df = spark.read.csv("sales.csv", header=True, inferSchema=True)
products_df = spark.read.csv("products.csv", header=True, inferSchema=True)

print("Schema of sales.csv:")
sales_df.printSchema()
print("Columns:", sales_df.columns)
print("Row count:", sales_df.count())
sales_df.show()
```

#### Output

```
=====
TASK 1: Read CSV and display schema
=====
```

```
Schema of sales.csv:
```

```

root
|-- sale_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: double (nullable = true)
|-- region: string (nullable = true)
|-- sale_date: date (nullable = true)
Columns: ['sale_id', 'product_id', 'quantity', 'unit_price', 'region', 'sale_date']
Row count: 12
+-----+-----+-----+-----+-----+
|sale_id|product_id|quantity|unit_price|region| sale_date|
+-----+-----+-----+-----+-----+
| 1| 101| 2| 500.0| North|2026-01-05|
| 2| 102| 1| 1500.0| South|2026-01-06|
| 3| 103| 3| 200.0| East|2026-01-07|
| 4| 101| 1| 500.0| West|2026-01-08|
| 5| 104| 2| 12000.0| North|2026-01-09|
| 5| 104| 2| 12000.0| North|2026-01-09|
| 6| 105| 5| 80.0| South|2026-01-10|
| 7| 102| 2| 1500.0| East|2026-01-11|
| 8| 106| 1| 55000.0| West|2026-01-12|
| 8| 106| 1| 55000.0| West|2026-01-12|
| 9| 103| 4| 200.0| North|2026-01-13|
| 10| 107| 1| 3000.0| South|2026-01-14|
+-----+-----+-----+-----+-----+

```

## TASK 2: Filtering, grouping and aggregation

### Code

```

print("=" * 60)
print("TASK 2: Filter, group and aggregate")
print("=" * 60)

# Derive total_amount
sales_df = sales_df.withColumn("total_amount", F.col("quantity") * F.col("unit_price"))

# Remove exact duplicates first so filters and aggregates are not inflated
# (Task 3 below demonstrates this step in detail)
clean_df = sales_df.dropDuplicates()

# Filtering
print("Sales with total_amount > 1000:")
clean_df.filter(F.col("total_amount") > 1000).show()

print("Sales in North region with quantity >= 2:")
clean_df.filter((F.col("region") == "North") & (F.col("quantity") >= 2)).show()

# Grouping + aggregation
print("Region-wise summary:")
clean_df.groupBy("region").agg(
    F.count("*").alias("num_sales"),
    F.sum("total_amount").alias("total_revenue"),
    F.avg("total_amount").alias("avg_sale"),
    F.max("total_amount").alias("max_sale"),
).orderBy(F.col("total_revenue").desc()).show()

```

### Output

```

=====
TASK 2: Filter, group and aggregate
=====

Sales with total_amount > 1000:
+-----+-----+-----+-----+-----+-----+
|sale_id|product_id|quantity|unit_price|region| sale_date|total_amount|
+-----+-----+-----+-----+-----+-----+
| 8| 106| 1| 55000.0| West|2026-01-12| 55000.0|
| 2| 102| 1| 1500.0| South|2026-01-06| 1500.0|
| 5| 104| 2| 12000.0| North|2026-01-09| 24000.0|
| 10| 107| 1| 3000.0| South|2026-01-14| 3000.0|
| 7| 102| 2| 1500.0| East|2026-01-11| 3000.0|
+-----+-----+-----+-----+-----+-----+

Sales in North region with quantity >= 2:
+-----+-----+-----+-----+-----+

```

| sale_id | product_id | quantity | unit_price | region | sale_date  | total_amount |
|---------|------------|----------|------------|--------|------------|--------------|
| 1       | 101        | 2        | 500.0      | North  | 2026-01-05 | 1000.0       |
| 5       | 104        | 2        | 12000.0    | North  | 2026-01-09 | 24000.0      |
| 9       | 103        | 4        | 200.0      | North  | 2026-01-13 | 800.0        |

Region-wise summary:

| region | num_sales | total_revenue | avg_sale           | max_sale |
|--------|-----------|---------------|--------------------|----------|
| West   | 2         | 55500.0       | 27750.0            | 55000.0  |
| North  | 3         | 25800.0       | 8600.0             | 24000.0  |
| South  | 3         | 4900.0        | 1633.3333333333333 | 3000.0   |
| East   | 2         | 3600.0        | 1800.0             | 3000.0   |

## TASK 3: Remove duplicate records

### Code

```
print("=" * 60)
print("TASK 3: Remove duplicates")
print("=" * 60)

print("Rows before:", sales_df.count())

dedup_df = sales_df.dropDuplicates() # removes fully identical rows
print("Rows after dropDuplicates():", dedup_df.count())

dedup_by_key = sales_df.dropDuplicates(["sale_id"]) # duplicates based on a key column
print("Rows after dropDuplicates(['sale_id']):", dedup_by_key.count())

print("Duplicate records that were found:")
(sales_df.groupBy("sale_id", "product_id", "quantity", "unit_price", "region", "sale_date")
 .count().filter(F.col("count") > 1).show())
```

### Output

```
TASK 3: Remove duplicates
```

```
Rows before: 12
```

```
Rows after dropDuplicates(): 10
```

```
Rows after dropDuplicates(['sale_id']): 10
```

```
Duplicate records that were found:
```

| sale_id | product_id | quantity | unit_price | region | sale_date  | count |
|---------|------------|----------|------------|--------|------------|-------|
| 8       | 106        | 1        | 55000.0    | West   | 2026-01-12 | 2     |
| 5       | 104        | 2        | 12000.0    | North  | 2026-01-09 | 2     |

## TASK 4: Join two datasets

### Code

```
print("=" * 60)
print("TASK 4: Join sales and products")
print("=" * 60)

inner_join = dedup_df.join(products_df, on="product_id", how="inner")
print("INNER JOIN (only matching products):", inner_join.count(), "rows")
inner_join.select("sale_id", "product_id", "product_name", "category",
                  "quantity", "total_amount").orderBy("sale_id").show()

left_join = dedup_df.join(products_df, on="product_id", how="left")
print("LEFT JOIN (keeps sales even without a matching product):", left_join.count(), "rows")
left_join.select("sale_id", "product_id", "product_name", "category") \
 .orderBy("sale_id").show()
```

### Output

```
TASK 4: Join sales and products
```

INNER JOIN (only matching products): 9 rows

| sale_id | product_id | product_name | category    | quantity | total_amount |
|---------|------------|--------------|-------------|----------|--------------|
| 1       | 101        | Mouse        | Accessories | 2        | 1000.0       |
| 2       | 102        | Keyboard     | Accessories | 1        | 1500.0       |
| 3       | 103        | Pen Drive    | Accessories | 3        | 600.0        |
| 4       | 101        | Mouse        | Accessories | 1        | 500.0        |
| 5       | 104        | Monitor      | Electronics | 2        | 24000.0      |
| 6       | 105        | Notebook     | Stationery  | 5        | 400.0        |
| 7       | 102        | Keyboard     | Accessories | 2        | 3000.0       |
| 8       | 106        | Laptop       | Electronics | 1        | 55000.0      |
| 9       | 103        | Pen Drive    | Accessories | 4        | 800.0        |

LEFT JOIN (keeps sales even without a matching product): 10 rows

| sale_id | product_id | product_name | category    |
|---------|------------|--------------|-------------|
| 1       | 101        | Mouse        | Accessories |
| 2       | 102        | Keyboard     | Accessories |
| 3       | 103        | Pen Drive    | Accessories |
| 4       | 101        | Mouse        | Accessories |
| 5       | 104        | Monitor      | Electronics |
| 6       | 105        | Notebook     | Stationery  |
| 7       | 102        | Keyboard     | Accessories |
| 8       | 106        | Laptop       | Electronics |
| 9       | 103        | Pen Drive    | Accessories |
| 10      | 107        | NULL         | NULL        |

## TASK 5: Average sales by product category

### Code

```
print("=" * 60)
print("TASK 5: Average sales by product category")
print("=" * 60)

category_avg = (
    inner_join.groupBy("category")
    .agg(
        F.count("*").alias("num_sales"),
        F.round(F.avg("total_amount"), 2).alias("avg_sales"),
        F.round(F.sum("total_amount"), 2).alias("total_sales"),
    )
    .orderBy(F.col("avg_sales").desc())
)
category_avg.show()

# Same result using Spark SQL
inner_join.createOrReplaceTempView("sales_products")
print("Same result using Spark SQL:")
spark.sql("""
    SELECT category,
           ROUND(AVG(total_amount), 2) AS avg_sales
    FROM sales_products
    GROUP BY category
    ORDER BY avg_sales DESC
""").show()

spark.stop()
```

### Output

TASK 5: Average sales by product category

| category    | num_sales | avg_sales | total_sales |
|-------------|-----------|-----------|-------------|
| Electronics | 2         | 39500.0   | 79000.0     |
| Accessories | 6         | 1233.33   | 7400.0      |
| Stationery  | 1         | 400.0     | 400.0       |

```
+-----+-----+-----+-----+
```

Same result using Spark SQL:

```
+-----+-----+
```

```
| category|avg_sales|
```

```
+-----+-----+
```

```
|Electronics| 39500.0|
```

```
|Accessories| 1233.33|
```

```
|Stationery| 400.0|
```

```
+-----+-----+
```

## Practical 9: End-to-End ETL Pipeline and E-Commerce Data Integration

### Aim

To build an end-to-end pipeline from CSV through Pandas cleaning into a SQL database with reporting, and to design an ETL pipeline for e-commerce data (customers, orders, products, payments) that handles both historical and newly arriving data.

### Code

```
import pandas as pd
import numpy as np
import sqlite3, os, shutil
import matplotlib.pyplot as plt

shutil.rmtree("data9", ignore_errors=True)
os.makedirs("data9/historical")
os.makedirs("data9/incoming")
```

### TASK 1: CSV -> Pandas -> Cleaning -> SQL Database -> Report

### Code

```
print("=" * 60)
print("TASK 1: End-to-end pipeline (CSV -> Pandas -> Clean -> SQL -> Report)")
print("=" * 60)

pd.DataFrame([
    [1, "Mumbai", "Phone", 2, 20000, "2026-01-03"],
    [2, "delhi", "Phone", 1, 20000, "2026-01-04"],
    [3, "Mumbai", "Charger", 5, 500, "2026-01-05"],
    [3, "Mumbai", "Charger", 5, 500, "2026-01-05"], # duplicate
    [4, "Pune", "Earbuds", None, 1500, "2026-01-06"], # missing qty
    [5, "Delhi", "Charger", 3, 500, "2026-01-07"],
    [6, "pune", "Phone", 1, 20000, "2026-02-01"],
    [7, "Mumbai", "Earbuds", 2, 1500, "2026-02-02"],
    [8, "Delhi", "Earbuds", 4, 1500, "2026-13-45"], # invalid date
    [9, "Pune", "Charger", 10, 500, "2026-02-10"],
], columns=["txn_id", "store", "item", "qty", "unit_price", "txn_date"]
).to_csv("data9/retail_sales.csv", index=False)

# --- Extract
df = pd.read_csv("data9/retail_sales.csv")
print("Raw rows:", len(df))

# --- Clean
df["store"] = df["store"].str.strip().str.title()
df["item"] = df["item"].str.strip().str.title()
df = df.drop_duplicates()
print("After removing duplicates:", len(df))

df["txn_date"] = pd.to_datetime(df["txn_date"], format="%Y-%m-%d", errors="coerce")
df = df.dropna(subset=["qty", "txn_date"])
print("After removing missing/invalid values:", len(df))

# --- Transform
df["qty"] = df["qty"].astype(int)
df["revenue"] = df["qty"] * df["unit_price"]
df["txn_date"] = df["txn_date"].dt.strftime("%Y-%m-%d")
print("\nCleaned data:")
print(df)

# --- Load
conn_a = sqlite3.connect("part_a_retail.db")
df.to_sql("retail_sales", conn_a, if_exists="replace", index=False)

# --- Report (SQL)
store_rev = pd.read_sql("""
    SELECT store, COUNT(*) AS transactions, SUM(revenue) AS total_revenue
    FROM retail_sales GROUP BY store ORDER BY total_revenue DESC""", conn_a)
item_rev = pd.read_sql("""
    SELECT item, SUM(qty) AS units_sold, SUM(revenue) AS total_revenue
```

```

FROM retail_sales GROUP BY item ORDER BY total_revenue DESC""", conn_a)
month_rev = pd.read_sql("""
SELECT SUBSTR(txn_date, 1, 7) AS month, SUM(revenue) AS total_revenue
FROM retail_sales GROUP BY month ORDER BY month""", conn_a)

print("\nREPORT 1: Revenue by store\n", store_rev)
print("\nREPORT 2: Revenue by item\n", item_rev)
print("\nREPORT 3: Revenue by month\n", month_rev)

store_rev.plot.bar(x="store", y="total_revenue", legend=False,
                  title="Task 1: Revenue by Store", figsize=(6, 4))
plt.ylabel("Revenue")
plt.tight_layout()
plt.show()

store_rev.to_csv("retail_report.csv", index=False)
conn_a.close()

```

## Output

TASK 1: End-to-end pipeline (CSV -> Pandas -> Clean -> SQL -> Report)

Raw rows: 10

After removing duplicates: 9

After removing missing/invalid values: 7

Cleaned data:

|   | txn_id | store  | item    | qty | unit_price | txn_date   | revenue |
|---|--------|--------|---------|-----|------------|------------|---------|
| 0 | 1      | Mumbai | Phone   | 2   | 20000      | 2026-01-03 | 40000   |
| 1 | 2      | Delhi  | Phone   | 1   | 20000      | 2026-01-04 | 20000   |
| 2 | 3      | Mumbai | Charger | 5   | 500        | 2026-01-05 | 2500    |
| 5 | 5      | Delhi  | Charger | 3   | 500        | 2026-01-07 | 1500    |
| 6 | 6      | Pune   | Phone   | 1   | 20000      | 2026-02-01 | 20000   |
| 7 | 7      | Mumbai | Earbuds | 2   | 1500       | 2026-02-02 | 3000    |
| 9 | 9      | Pune   | Charger | 10  | 500        | 2026-02-10 | 5000    |

REPORT 1: Revenue by store

|   | store  | transactions | total_revenue |
|---|--------|--------------|---------------|
| 0 | Mumbai | 3            | 45500         |
| 1 | Pune   | 2            | 25000         |
| 2 | Delhi  | 2            | 21500         |

REPORT 2: Revenue by item

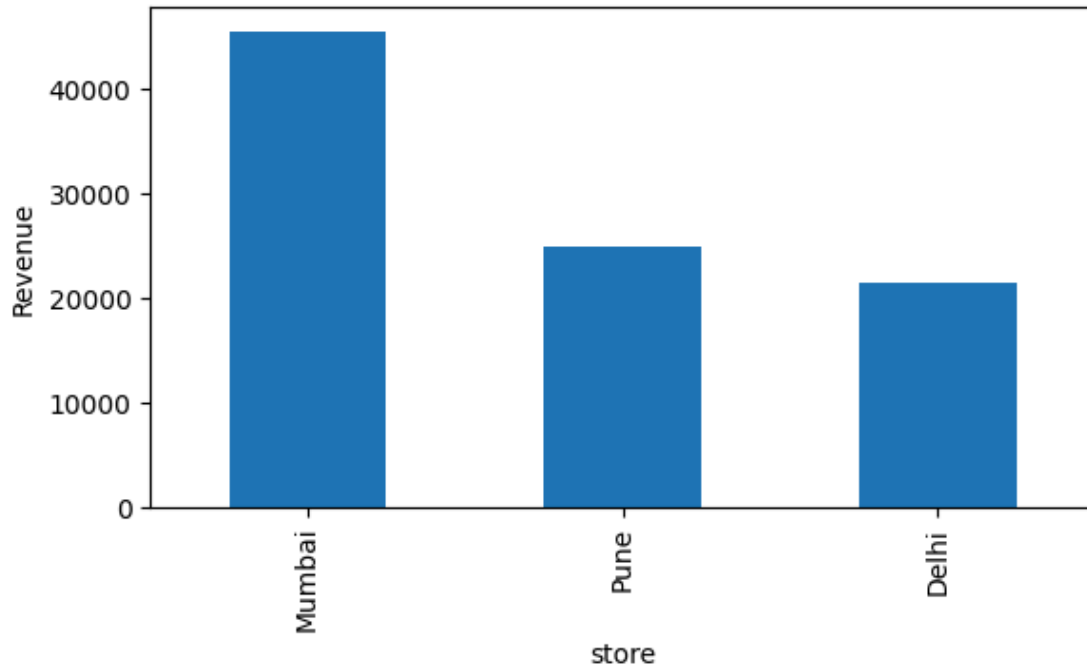
|   | item    | units_sold | total_revenue |
|---|---------|------------|---------------|
| 0 | Phone   | 4          | 80000         |
| 1 | Charger | 18         | 9000          |
| 2 | Earbuds | 2          | 3000          |

REPORT 3: Revenue by month

|   | month   | total_revenue |
|---|---------|---------------|
| 0 | 2026-01 | 64000         |
| 1 | 2026-02 | 28000         |

## Output (chart)

Task 1: Revenue by Store



## TASK 2 + 3 SETUP: E-commerce database

### Code

```
# ----- Historical files -----
pd.DataFrame([
    [1, "Asha Rao", "ASHA@mail.com", "Mumbai"],
    [2, "rohan das", "rohan@mail.com", "Pune"],
    [2, "rohan das", "rohan@mail.com", "Pune"], # duplicate
    [3, "Meera Iyer", "meera@mail.com", "chennai"],
    [4, "Karan Shah", "karan@mail.com", "Delhi"],
    [5, "Neha Gupta", None, "Jaipur"], # missing email
    [6, "Vikram Singh", "vikram-at-mail.com", "Kolkata"], # invalid email
    [7, "Priya Nair", "priya@mail.com", "Kochi"],
], columns=["customer_id", "name", "email", "city"])
.to_csv("data9/historical/customers.csv", index=False)

pd.DataFrame([
    [101, "Laptop", "Electronics", 55000],
    [102, "Mouse", "Accessories", 500],
    [103, "Keyboard", "Accessories", 1500],
    [104, "Monitor", "Electronics", 12000],
    [105, "Notebook", "Stationery", 80],
    [106, "Pen Drive", "Accessories", -200], # invalid price
], columns=["product_id", "product_name", "category", "price"])
.to_csv("data9/historical/products.csv", index=False)

pd.DataFrame([
    [1, 1, 101, 1, "2026-01-05"],
    [2, 2, 102, 2, "2026-01-10"],
    [3, 3, 103, 1, "2026-01-15"],
    [4, 4, 104, 1, "2026-01-20"],
    [5, 7, 105, 5, "2026-01-22"],
    [6, 2, 101, 1, "2026-02-03"],
    [7, 99, 102, 1, "2026-02-05"], # unknown customer
    [8, 1, 106, 1, "2026-02-08"], # product 106 was rejected
    [9, 3, 102, -1, "2026-02-10"], # negative quantity
    [10, 4, 105, 2, "bad-date"], # invalid date
    [11, 5, 103, 1, "2026-02-12"], # customer 5 was rejected
], columns=["order_id", "customer_id", "product_id", "quantity", "order_date"])
.to_csv("data9/historical/orders.csv", index=False)
```

```

pd.DataFrame([
    [1, 1, 55000, "UPI", "SUCCESS", "2026-01-05"],
    [2, 2, 1000, "Card", "SUCCESS", "2026-01-10"],
    [3, 3, 1500, "COD", "SUCCESS", "2026-01-16"],
    [4, 4, 12000, "card", "FAILED", "2026-01-20"],
    [5, 5, 400, "UPI", "SUCCESS", "2026-01-22"],
    [6, 6, 55000, "upi", "SUCCESS", "2026-02-03"],
    [7, 7, 500, "UPI", "SUCCESS", "2026-02-05"], # order 7 was rejected
], columns=["payment_id", "order_id", "amount", "method", "status", "payment_date"])
.to_csv("data9/historical/payments.csv", index=False)

# ----- Newly arriving files -----
pd.DataFrame([
    [8, "Sanjay Kulkarni", "sanjay@mail.com", "Nashik"], # new customer
    [1, "Asha Rao", "asha@mail.com", "Thane"], # existing, city changed
], columns=["customer_id", "name", "email", "city"])
.to_csv("data9/incoming/customers.csv", index=False)

pd.DataFrame([
    [107, "Headphones", "Accessories", 2500],
], columns=["product_id", "product_name", "category", "price"])
.to_csv("data9/incoming/products.csv", index=False)

pd.DataFrame([
    [12, 8, 107, 2, "2026-03-02"],
    [13, 1, 104, 1, "2026-03-05"],
    [14, 2, 102, 3, "2026-03-09"],
    [6, 2, 101, 1, "2026-02-03"], # re-sent, already loaded
    [15, 77, 102, 1, "2026-03-10"], # unknown customer
], columns=["order_id", "customer_id", "product_id", "quantity", "order_date"])
.to_csv("data9/incoming/orders.csv", index=False)

pd.DataFrame([
    [8, 12, 5000, "UPI", "SUCCESS", "2026-03-02"],
    [9, 13, 12000, "CARD", "SUCCESS", "2026-03-05"],
    [10, 14, 1500, "COD", "SUCCESS", "2026-03-09"],
    [6, 6, 55000, "UPI", "SUCCESS", "2026-02-03"], # re-sent, already loaded
    [11, 15, 500, "UPI", "SUCCESS", "2026-03-10"], # order 15 was rejected
], columns=["payment_id", "order_id", "amount", "method", "status", "payment_date"])
.to_csv("data9/incoming/payments.csv", index=False)

# ----- Target database -----
DB = "ecommerce.db"
if os.path.exists(DB):
    os.remove(DB)
conn = sqlite3.connect(DB)
conn.executescript("""
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT, email TEXT, city TEXT);
CREATE TABLE products (product_id INTEGER PRIMARY KEY, product_name TEXT, category TEXT, price REAL);
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY, customer_id INTEGER, product_id INTEGER,
    quantity INTEGER, order_date TEXT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id));
CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY, order_id INTEGER, amount REAL,
    method TEXT, status TEXT, payment_date TEXT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id));
CREATE TABLE rejected_records (source TEXT, batch TEXT, record TEXT, reason TEXT);
CREATE TABLE batch_log (batch TEXT PRIMARY KEY, loaded_at TEXT, rejected INTEGER);
""")

# ----- Helper functions -----
def split_valid(df, rules):
    """Apply validation rules. Returns (valid, invalid, reasons_for_invalid)."""
    reasons = pd.Series("", index=df.index)
    for msg, cond in rules.items():
        reasons.loc[cond] = reasons.loc[cond] + msg + "; "
    bad = reasons != ""
    return df[~bad].copy(), df[bad].copy(), reasons[bad]

def log_rejected(bad, reasons, source, batch):

```

```

rows = [(source, batch, r.to_json(), reasons[i]) for i, r in bad.iterrows()]
if rows:
    conn.executemany("INSERT INTO rejected_records VALUES (?, ?, ?, ?)", rows)

def insert(table, df, cols, verb):
    if df.empty:
        return 0
    rows = df[cols].astype(object).values.tolist()
    q = f"{verb} INTO {table} ({','.join(cols)}) VALUES ({','.join(['?'] * len(cols))})"
    before = conn.total_changes
    conn.executemany(q, rows)
    return conn.total_changes - before

def process(name, raw_n, df, rules, table, cols, cast, batch, verb):
    valid, bad, reasons = split_valid(df, rules)
    log_rejected(bad, reasons, name, batch)
    valid = valid.astype(cast)
    written = insert(table, valid, cols, verb)
    print(f"{name:<10} raw={raw_n:<3} dupes_removed={raw_n - len(df):<2} "
          f"rejected={len(bad):<2} valid={len(valid):<2} written={written:<2} "
          f"ignored_existing={len(valid) - written}")

def to_date(s):
    return pd.to_datetime(s, format="%Y-%m-%d", errors="coerce").dt.strftime("%Y-%m-%d")

```

## TASK 2: E-commerce ETL pipeline (one batch = 4 files)

### Code

```

def run_batch(folder, batch):
    print(f"\n---- {batch} ----")
    if conn.execute("SELECT 1 FROM batch_log WHERE batch=?", (batch,)).fetchone():
        print("Batch already processed -> skipped (pipeline is idempotent).")
        return

    # ---- CUSTOMERS (dimension: upsert so changed details update) ----
    c = pd.read_csv(f"{folder}/customers.csv")
    n = len(c)
    c["name"] = c["name"].str.strip().str.title()
    c["email"] = c["email"].str.strip().str.lower()
    c["city"] = c["city"].str.strip().str.title()
    c = c.drop_duplicates(subset=["customer_id"], keep="last")
    c_rules = {
        "missing email": c["email"].isna(),
        "invalid email": c["email"].notna() &
            ~c["email"].fillna("").str.match(r"^\w.+]+@[\w-]+\.[\w.]+$"),
    }
    process("customers", n, c, c_rules, "customers",
           ["customer_id", "name", "email", "city"],
           {"customer_id": int}, batch, "INSERT OR REPLACE")

    # ---- PRODUCTS (dimension: upsert) ----
    p = pd.read_csv(f"{folder}/products.csv")
    n = len(p)
    p["product_name"] = p["product_name"].str.strip().str.title()
    p["category"] = p["category"].str.strip().str.title()
    p["price"] = pd.to_numeric(p["price"], errors="coerce")
    p = p.drop_duplicates(subset=["product_id"], keep="last")
    p_rules = {
        "missing name": p["product_name"].isna(),
        "invalid price": p["price"].isna() | (p["price"] <= 0),
    }
    process("products", n, p, p_rules, "products",
           ["product_id", "product_name", "category", "price"],
           {"product_id": int, "price": float}, batch, "INSERT OR REPLACE")

    # ---- ORDERS (fact: insert new only) ----
    o = pd.read_csv(f"{folder}/orders.csv")
    n = len(o)
    for col in ["order_id", "customer_id", "product_id", "quantity"]:
        o[col] = pd.to_numeric(o[col], errors="coerce")

```

```

o["order_date_raw"] = o["order_date"]
o["order_date"] = to_date(o["order_date"])
o = o.drop_duplicates(subset=["order_id"])
cust_ids = [r[0] for r in conn.execute("SELECT customer_id FROM customers")]
prod_ids = [r[0] for r in conn.execute("SELECT product_id FROM products")]
o_rules = {
    "invalid quantity": o["quantity"].isna() | (o["quantity"] <= 0),
    "invalid date": o["order_date"].isna(),
    "unknown customer": ~o["customer_id"].isin(cust_ids),
    "unknown product": ~o["product_id"].isin(prod_ids),
}
process("orders", n, o, o_rules, "orders",
       ["order_id", "customer_id", "product_id", "quantity", "order_date"],
       {"order_id": int, "customer_id": int, "product_id": int, "quantity": int},
       batch, "INSERT OR IGNORE")

# ---- PAYMENTS (fact: insert new only) ----
pay = pd.read_csv(f"{folder}/payments.csv")
n = len(pay)
pay["amount"] = pd.to_numeric(pay["amount"], errors="coerce")
pay["method"] = pay["method"].str.strip().str.upper()
pay["status"] = pay["status"].str.strip().str.upper()
pay["payment_date_raw"] = pay["payment_date"]
pay["payment_date"] = to_date(pay["payment_date"])
pay = pay.drop_duplicates(subset=["payment_id"])
order_ids = [r[0] for r in conn.execute("SELECT order_id FROM orders")]
pay_rules = {
    "invalid amount": pay["amount"].isna() | (pay["amount"] <= 0),
    "invalid status": ~pay["status"].isin(["SUCCESS", "FAILED", "PENDING"]),
    "invalid date": pay["payment_date"].isna(),
    "unknown order": ~pay["order_id"].isin(order_ids),
}
process("payments", n, pay, pay_rules, "payments",
       ["payment_id", "order_id", "amount", "method", "status", "payment_date"],
       {"payment_id": int, "order_id": int, "amount": float},
       batch, "INSERT OR IGNORE")

rej = conn.execute("SELECT COUNT(*) FROM rejected_records WHERE batch=?", (batch,)).fetchone()[0]
conn.execute("INSERT INTO batch_log VALUES (?, datetime('now'), ?)", (batch, rej))
conn.commit()

def counts():
    return pd.read_sql("""
        SELECT (SELECT COUNT(*) FROM customers)      AS customers,
               (SELECT COUNT(*) FROM products)       AS products,
               (SELECT COUNT(*) FROM orders)          AS orders,
               (SELECT COUNT(*) FROM payments)        AS payments,
               (SELECT COUNT(*) FROM rejected_records) AS rejected""", conn)

print("\n" + "=" * 60)
print("TASK 2: E-commerce ETL - historical load")
print("=" * 60)
run_batch("data9/historical", "batch_1_historical")
print("\nRow counts after historical load:")
print(counts())

```

## Output

```
=====
TASK 2: E-commerce ETL - historical load
=====
```

```

----- batch_1_historical -----
customers raw=8  dupes_removed=1  rejected=2  valid=5  written=5  ignored_existing=0
products  raw=6  dupes_removed=0  rejected=1  valid=5  written=5  ignored_existing=0
orders    raw=11 dupes_removed=0  rejected=5  valid=6  written=6  ignored_existing=0
payments  raw=7  dupes_removed=0  rejected=1  valid=6  written=6  ignored_existing=0
Row counts after historical load:
  customers  products  orders  payments  rejected
0         5         5         6         6         9

```

## TASK 3: Handle newly arriving data alongside history

## Code

```
print("\n" + "=" * 60)
print("TASK 3: Newly arriving data (incremental load)")
print("=" * 60)
run_batch("data9/incoming", "batch_2_incoming")
print("\nRow counts after incremental load:")
print(counts())

print("\nRe-running the same incoming batch:")
run_batch("data9/incoming", "batch_2_incoming")

print("\nCustomers table (note Asha's city changed to Thane, Sanjay added):")
print(pd.read_sql("SELECT * FROM customers ORDER BY customer_id", conn))

print("\nRejected records log:")
pd.set_option("display.max_colwidth", 60)
print(pd.read_sql("SELECT source, batch, reason FROM rejected_records", conn))

print("\nBatch log:")
print(pd.read_sql("SELECT * FROM batch_log", conn))
```

## Output

```
=====
TASK 3: Newly arriving data (incremental load)
=====

---- batch_2_incoming ----
customers raw=2 dupes_removed=0 rejected=0 valid=2 written=2 ignored_existing=0
products raw=1 dupes_removed=0 rejected=0 valid=1 written=1 ignored_existing=0
orders raw=5 dupes_removed=0 rejected=1 valid=4 written=3 ignored_existing=1
payments raw=5 dupes_removed=0 rejected=1 valid=4 written=3 ignored_existing=1
Row counts after incremental load:
  customers products orders payments rejected
0         6         6         9         9         11
Re-running the same incoming batch:
---- batch_2_incoming ----
Batch already processed -> skipped (pipeline is idempotent).
Customers table (note Asha's city changed to Thane, Sanjay added):
  customer_id  name      email      city
0          1  Asha Rao  asha@mail.com  Thane
1          2  Rohan Das  rohan@mail.com  Pune
2          3  Meera Iyer  meera@mail.com  Chennai
3          4  Karan Shah  karan@mail.com  Delhi
4          7  Priya Nair  priya@mail.com  Kochi
5          8  Sanjay Kulkarni  sanjay@mail.com  Nashik
Rejected records log:
  source      batch      reason
0 customers batch_1_historical  missing email;
1 customers batch_1_historical  invalid email;
2 products batch_1_historical  invalid price;
3 orders batch_1_historical  unknown customer;
4 orders batch_1_historical  unknown product;
5 orders batch_1_historical  invalid quantity;
6 orders batch_1_historical  invalid date;
7 orders batch_1_historical  unknown customer;
8 payments batch_1_historical  unknown order;
9 orders batch_2_incoming  unknown customer;
10 payments batch_2_incoming  unknown order;
Batch log:
      batch      loaded_at rejected
0 batch_1_historical 2026-09-28 14:09:53    9
1 batch_2_incoming 2026-09-28 14:09:54    2
```

## Code

```
# =====
# FINAL REPORTS (SUCCESSFUL payments only)
# =====

print("\n" + "=" * 60)
print("E-COMMERCE REPORTS")
print("=" * 60)

cat = pd.read_sql("""
  SELECT pr.category, COUNT(DISTINCT o.order_id) AS orders, ROUND(SUM(p.amount), 2) AS revenue
```

```

FROM payments p
JOIN orders o ON p.order_id = o.order_id
JOIN products pr ON o.product_id = pr.product_id
WHERE p.status = 'SUCCESS'
GROUP BY pr.category ORDER BY revenue DESC""", conn)
print("\nRevenue by category:\n", cat)

top = pd.read_sql("""
SELECT c.name, c.city, ROUND(SUM(p.amount), 2) AS total_spent
FROM payments p
JOIN orders o ON p.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
WHERE p.status = 'SUCCESS'
GROUP BY c.customer_id ORDER BY total_spent DESC""", conn)
print("\nTop customers:\n", top)

mon = pd.read_sql("""
SELECT SUBSTR(payment_date, 1, 7) AS month, ROUND(SUM(amount), 2) AS revenue
FROM payments WHERE status = 'SUCCESS'
GROUP BY month ORDER BY month""", conn)
print("\nMonthly revenue:\n", mon)

pay_status = pd.read_sql("""
SELECT status, COUNT(*) AS payments, ROUND(SUM(amount), 2) AS amount
FROM payments GROUP BY status""", conn)
print("\nPayment status summary:\n", pay_status)

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
cat.plot.bar(x="category", y="revenue", ax=ax[0], legend=False, title="Revenue by Category")
mon.plot.bar(x="month", y="revenue", ax=ax[1], legend=False, title="Monthly Revenue")
plt.tight_layout()
plt.show()

top.to_csv("ecommerce_top_customers.csv", index=False)
conn.close()

```

## Output

### E-COMMERCE REPORTS

#### Revenue by category:

|   | category    | orders | revenue  |
|---|-------------|--------|----------|
| 0 | Electronics | 3      | 122000.0 |
| 1 | Accessories | 4      | 9000.0   |
| 2 | Stationery  | 1      | 400.0    |

#### Top customers:

|   | name            | city    | total_spent |
|---|-----------------|---------|-------------|
| 0 | Asha Rao        | Thane   | 67000.0     |
| 1 | Rohan Das       | Pune    | 57500.0     |
| 2 | Sanjay Kulkarni | Nashik  | 5000.0      |
| 3 | Meera Iyer      | Chennai | 1500.0      |
| 4 | Priya Nair      | Kochi   | 400.0       |

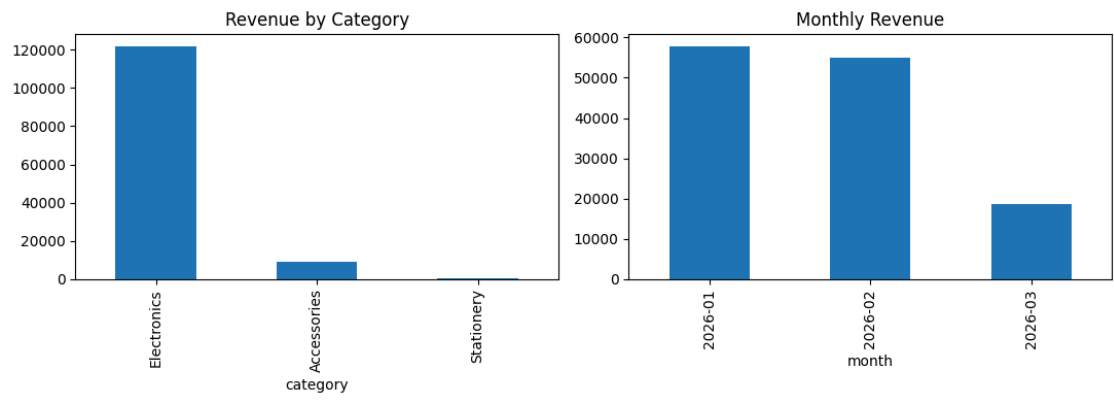
#### Monthly revenue:

|   | month   | revenue |
|---|---------|---------|
| 0 | 2026-01 | 57900.0 |
| 1 | 2026-02 | 55000.0 |
| 2 | 2026-03 | 18500.0 |

#### Payment status summary:

|   | status  | payments | amount   |
|---|---------|----------|----------|
| 0 | FAILED  | 1        | 12000.0  |
| 1 | SUCCESS | 8        | 131400.0 |

## Output (chart)



# Practical 10: Mini Project – Retail Analytics Data Warehouse

## Aim

To design and implement an end-to-end data engineering solution covering data ingestion, cleaning, transformation, storage, data warehouse (star schema) design, and reporting.

## Code

```
import pandas as pd
import numpy as np
import sqlite3, os, shutil, json
import matplotlib.pyplot as plt
```

```
# =====
# STEP 0: SETUP
# =====
shutil.rmtree("mini_project", ignore_errors=True)
os.makedirs("mini_project/raw")
os.makedirs("mini_project/reports")
DB = "mini_project/retail_dw.db"
rng = np.random.default_rng(42)
```

## STEP 1: DATA SOURCE (simulated raw files with deliberate dirt)

## Code

```
cities = ["Mumbai", "Pune", "Delhi", "Chennai", "Kolkata", "Jaipur"]

# ---- customers
cust = pd.DataFrame({
    "customer_id": range(1, 31),
    "name": [f"Customer {i}" for i in range(1, 31)],
    "email": [f"cust{i}@mail.com" for i in range(1, 31)],
    "city": rng.choice(cities, 30).tolist(),
    "signup_date": pd.date_range("2024-01-01", periods=30, freq="10D").strftime("%Y-%m-%d"),
})
cust.loc[2, "name"] = " customer 3 " # needs trimming/casing
cust.loc[6, "email"] = None # missing email
cust.loc[8, "email"] = "cust9-at-mail.com" # invalid email
cust.loc[13, "city"] = " pune" # needs trimming/casing
cust = pd.concat([cust, cust.iloc[[4]], ignore_index=True) # exact duplicate

# ---- products
prod = pd.DataFrame([
    [1, "Laptop", "Electronics", 55000], [2, "Smartphone", "Electronics", 22000],
    [3, "Monitor", "Electronics", 12000], [4, "Headphones", "Accessories", 2500],
    [5, "Mouse", "Accessories", 500], [6, "Keyboard", "Accessories", 1500],
    [7, "Backpack", "Fashion", 1800], [8, "Sneakers", "Fashion", 3200],
    [9, "T-Shirt", "Fashion", 700], [10, "Notebook", "Stationery", 80],
    [11, "Desk Lamp", "Home", 900], [12, "Water Bottle", "Home", 350],
], columns=["product_id", "product_name", "category", "unit_price"])
price_map = dict(zip(prod["product_id"], prod["unit_price"]))
prod.loc[4, "product_name"] = " mouse "
prod = pd.concat([prod, prod.iloc[[0]], # duplicate
                  pd.DataFrame([[13, "Broken Item", "Home", -100]], columns=prod.columns), # invalid price
                  ignore_index=True)

# ---- orders (200 good rows + dirty rows)
n = 200
orders = pd.DataFrame({
    "order_id": range(1, n + 1),
    "customer_id": rng.integers(1, 31, n),
    "product_id": rng.integers(1, 13, n),
    "quantity": rng.integers(1, 6, n),
    "discount_pct": rng.choice([0, 0, 5, 10], n),
    "order_date": (pd.to_datetime("2025-01-01")
                   + pd.to_timedelta(rng.integers(0, 365, n), unit="D")).strftime("%Y-%m-%d"),
})

# ---- payments (one per good order)
amounts = [round(q * price_map[p] * (1 - d / 100), 2)
            for q, p, d in zip(orders["quantity"], orders["product_id"], orders["discount_pct"])]
```

```

payments = pd.DataFrame({
    "payment_id": orders["order_id"],
    "order_id": orders["order_id"],
    "amount": amounts,
    "method": rng.choice(["UPI", "Card", "COD", "NetBanking", "upi", "card"], n).tolist(),
    "status": rng.choice(["SUCCESS", "FAILED", "PENDING"], n, p=[0.88, 0.08, 0.04]).tolist(),
    "payment_date": orders["order_date"],
})

bad_orders = pd.DataFrame([
    [201, 5, 3, -1, 0, "2025-03-01"], [202, 6, 4, 0, 0, "2025-03-02"], # invalid quantity
    [203, 7, 5, -3, 0, "2025-03-03"], [204, 8, 6, 0, 5, "2025-03-04"],
    [205, 9, 2, 1, 0, "2025-13-40"], [206, 10, 3, 1, 0, "not-a-date"], # invalid date
    [207, 11, 4, 2, 0, ""],
    [208, 999, 2, 1, 0, "2025-04-01"], [209, 998, 3, 1, 0, "2025-04-02"], # unknown customer
    [210, 997, 4, 1, 0, "2025-04-03"],
    [211, 5, 888, 1, 0, "2025-05-01"], [212, 6, 887, 1, 0, "2025-05-02"], # unknown product
], columns=orders.columns)
bad_pay = pd.DataFrame([
    [5000, 5000, 1000, "UPI", "SUCCESS", "2025-06-01"], # unknown order
    [5001, 5001, 2000, "card", "SUCCESS", "2025-06-02"],
    [5002, 5002, -50, "UPI", "SUCCESS", "2025-06-03"], # bad amount + unknown order
], columns=payments.columns)

orders = pd.concat([orders, bad_orders, orders.iloc[[9, 49, 99, 149, 199]]], ignore_index=True) # 5 dupes
payments = pd.concat([payments, bad_pay, payments.iloc[[9]]], ignore_index=True) # 1 dupe

cust.to_csv("mini_project/raw/customers.csv", index=False)
prod.to_csv("mini_project/raw/products.csv", index=False)
orders.to_csv("mini_project/raw/orders.csv", index=False)
payments.to_csv("mini_project/raw/payments.csv", index=False)
print("Raw files created:", os.listdir("mini_project/raw"))

```

## Output

```
Raw files created: ['orders.csv', 'products.csv', 'payments.csv', 'customers.csv']
```

## STEP 2: INGESTION -> STAGING (loaded as-is, everything TEXT)

### Code

```

conn = sqlite3.connect(DB)
print("\n" + "=" * 60 + "\nSTEP 2: INGESTION (raw CSV -> staging tables)\n" + "=" * 60)
for name in ["customers", "products", "orders", "payments"]:
    raw_df = pd.read_csv(f"mini_project/raw/{name}.csv", dtype=str)
    raw_df.to_sql(f"stg_{name}", conn, if_exists="replace", index=False)
    print(f"stg_{name}<10} {len(raw_df)} rows")

```

## Output

```
=====
STEP 2: INGESTION (raw CSV -> staging tables)
=====
```

```

stg_customers 31 rows
stg_products  14 rows
stg_orders    217 rows
stg_payments  204 rows

```

## STEP 3: CLEANING & VALIDATION

### Code

```

print("\n" + "=" * 60 + "\nSTEP 3: CLEANING & VALIDATION\n" + "=" * 60)

dq, rejected = [], []
EMAIL_RE = r"^[w.-]+@[w.-]+\.[w.-]+$"

def to_date(s):
    return pd.to_datetime(s, format="%Y-%m-%d", errors="coerce").dt.strftime("%Y-%m-%d")

def apply_rules(df, rules):
    reasons = pd.Series("", index=df.index)

```

```

for msg, cond in rules.items():
    reasons.loc[cond] = reasons.loc[cond] + msg + "; "
bad = reasons != ""
return df[~bad].copy(), df[bad].copy(), reasons[bad]

def finish(entity, raw_n, dupes, repaired, valid, bad, why):
    for i in bad.index:
        rejected.append((entity, bad.loc[i].to_json(), why[i]))
    dq.append({"entity": entity, "raw_rows": raw_n, "duplicates_removed": dupes,
              "values_repaired": repaired, "rejected": len(bad), "loaded": len(valid)})

# ---- customers
c = pd.read_sql("SELECT * FROM stg_customers", conn)
raw_n = len(c)
c["customer_id"] = pd.to_numeric(c["customer_id"], errors="coerce")
c["name"] = c["name"].str.strip().str.title()
c["city"] = c["city"].str.strip().str.title()
c["email"] = c["email"].str.strip().str.lower()
c["signup_date"] = to_date(c["signup_date"])
c = c.drop_duplicates(subset="customer_id", keep="last").reset_index(drop=True)
dupes = raw_n - len(c)
bad_email = c["email"].isna() | ~c["email"].fillna("").str.match(EMAIL_RE)
repaired = int(bad_email.sum())
c.loc[bad_email, "email"] = "unknown@unknown.com" # repair instead of reject
cust_ok, cust_bad, why = apply_rules(c, {
    "missing customer_id": c["customer_id"].isna(),
    "missing name": c["name"].isna()})
cust_ok["customer_id"] = cust_ok["customer_id"].astype(int)
finish("customers", raw_n, dupes, repaired, cust_ok, cust_bad, why)

# ---- products
p = pd.read_sql("SELECT * FROM stg_products", conn)
raw_n = len(p)
p["product_id"] = pd.to_numeric(p["product_id"], errors="coerce")
p["unit_price"] = pd.to_numeric(p["unit_price"], errors="coerce")
p["product_name"] = p["product_name"].str.strip().str.title()
p["category"] = p["category"].str.strip().str.title()
p = p.drop_duplicates(subset="product_id", keep="last").reset_index(drop=True)
dupes = raw_n - len(p)
prod_ok, prod_bad, why = apply_rules(p, {
    "missing product_id": p["product_id"].isna(),
    "missing name": p["product_name"].isna(),
    "invalid price": p["unit_price"].isna() | (p["unit_price"] <= 0)})
prod_ok = prod_ok.astype({"product_id": int, "unit_price": float})
prod_ok["unit_cost"] = (prod_ok["unit_price"] * 0.70).round(2) # assumption: cost = 70% of price
finish("products", raw_n, dupes, 0, prod_ok, prod_bad, why)

# ---- orders
o = pd.read_sql("SELECT * FROM stg_orders", conn)
raw_n = len(o)
for col in ["order_id", "customer_id", "product_id", "quantity", "discount_pct"]:
    o[col] = pd.to_numeric(o[col], errors="coerce")
o["discount_pct"] = o["discount_pct"].fillna(0)
o["order_date"] = to_date(o["order_date"])
o = o.drop_duplicates(subset="order_id").reset_index(drop=True)
dupes = raw_n - len(o)
orders_ok, orders_bad, why = apply_rules(o, {
    "missing order_id": o["order_id"].isna(),
    "invalid quantity": o["quantity"].isna() | (o["quantity"] <= 0),
    "invalid date": o["order_date"].isna(),
    "unknown customer": ~o["customer_id"].isin(cust_ok["customer_id"]),
    "unknown product": ~o["product_id"].isin(prod_ok["product_id"])})
orders_ok = orders_ok.astype({"order_id": int, "customer_id": int, "product_id": int,
                              "quantity": int, "discount_pct": float})
finish("orders", raw_n, dupes, 0, orders_ok, orders_bad, why)

# ---- payments
pay = pd.read_sql("SELECT * FROM stg_payments", conn)
raw_n = len(pay)
for col in ["payment_id", "order_id", "amount"]:
    pay[col] = pd.to_numeric(pay[col], errors="coerce")
pay["method"] = pay["method"].str.strip().str.upper()

```

```

pay["status"] = pay["status"].str.strip().str.upper()
pay["payment_date"] = to_date(pay["payment_date"])
pay = pay.drop_duplicates(subset="payment_id").reset_index(drop=True)
dupes = raw_n - len(pay)
pay_ok, pay_bad, why = apply_rules(pay, {
    "invalid amount": pay["amount"].isna() | (pay["amount"] <= 0),
    "invalid status": ~pay["status"].isin(["SUCCESS", "FAILED", "PENDING"]),
    "invalid date": pay["payment_date"].isna(),
    "unknown order": ~pay["order_id"].isin(orders_ok["order_id"])}
pay_ok = pay_ok.astype({"payment_id": int, "order_id": int, "amount": float})
finish("payments", raw_n, dupes, 0, pay_ok, pay_bad, why)

# Save cleaned layer + data-quality log
for name, df in [("customers", cust_ok), ("products", prod_ok),
    ("orders", orders_ok), ("payments", pay_ok)]:
    df.to_sql(f"cln_{name}", conn, if_exists="replace", index=False)
rej_df = pd.DataFrame(rejected, columns=["entity", "record_json", "reason"])
rej_df.to_sql("rejected_records", conn, if_exists="replace", index=False)
dq_df = pd.DataFrame(dq)
dq_df.to_sql("etl_dq_summary", conn, if_exists="replace", index=False)

print("DATA QUALITY SUMMARY")
print(dq_df.to_string(index=False))
print("\nRejected records by reason:")
print(rej_df.groupby(["entity", "reason"]).size().rename("count"))

```

## Output

### STEP 3: CLEANING & VALIDATION

#### DATA QUALITY SUMMARY

| entity    | raw_rows | duplicates_removed | values_repaired | rejected | loaded |
|-----------|----------|--------------------|-----------------|----------|--------|
| customers | 31       | 1                  | 2               | 0        | 30     |
| products  | 14       | 1                  | 0               | 1        | 12     |
| orders    | 217      | 5                  | 0               | 12       | 200    |
| payments  | 204      | 1                  | 0               | 3        | 200    |

#### Rejected records by reason:

| entity   | reason                         | count |
|----------|--------------------------------|-------|
| orders   | invalid date;                  | 3     |
|          | invalid quantity;              | 4     |
|          | unknown customer;              | 3     |
|          | unknown product;               | 2     |
| payments | invalid amount; unknown order; | 1     |
|          | unknown order;                 | 2     |
| products | invalid price;                 | 1     |

Name: count, dtype: int64

## STEP 4: DATA WAREHOUSE (star schema)

### Code

```

print("\n" + "=" * 60 + "\nSTEP 4: DATA WAREHOUSE (star schema)\n" + "=" * 60)

conn.executescript("""
DROP VIEW IF EXISTS vw_sales_report;
DROP TABLE IF EXISTS fact_sales;
DROP TABLE IF EXISTS dim_date;
DROP TABLE IF EXISTS dim_customer;
DROP TABLE IF EXISTS dim_product;
DROP TABLE IF EXISTS dim_payment;

CREATE TABLE dim_date (
    date_key INTEGER PRIMARY KEY, full_date TEXT, year INTEGER, quarter INTEGER,
    month INTEGER, month_name TEXT, day INTEGER, day_name TEXT, is_weekend INTEGER);
CREATE TABLE dim_customer (
    customer_key INTEGER PRIMARY KEY AUTOINCREMENT, customer_id INTEGER UNIQUE,
    name TEXT, email TEXT, city TEXT, signup_date TEXT);
CREATE TABLE dim_product (
    product_key INTEGER PRIMARY KEY AUTOINCREMENT, product_id INTEGER UNIQUE,
    product_name TEXT, category TEXT, unit_price REAL, unit_cost REAL);
CREATE TABLE dim_payment (
    payment_key INTEGER PRIMARY KEY AUTOINCREMENT, method TEXT, status TEXT,

```

```

        UNIQUE (method, status));
CREATE TABLE fact_sales (
    sales_key INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER UNIQUE,
    date_key INTEGER, customer_key INTEGER, product_key INTEGER, payment_key INTEGER,
    quantity INTEGER, gross_amount REAL, discount_amount REAL, net_amount REAL,
    cost_amount REAL, profit REAL, amount_paid REAL,
    FOREIGN KEY (date_key) REFERENCES dim_date(date_key),
    FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key),
    FOREIGN KEY (product_key) REFERENCES dim_product(product_key),
    FOREIGN KEY (payment_key) REFERENCES dim_payment(payment_key));
""")

# ---- dim_date (full calendar for 2025)
d = pd.DataFrame({"full_date": pd.date_range("2025-01-01", "2025-12-31")})
d["date_key"] = d["full_date"].dt.strftime("%Y%m%d").astype(int)
d["year"] = d["full_date"].dt.year
d["quarter"] = d["full_date"].dt.quarter
d["month"] = d["full_date"].dt.month
d["month_name"] = d["full_date"].dt.strftime("%b")
d["day"] = d["full_date"].dt.day
d["day_name"] = d["full_date"].dt.day_name()
d["is_weekend"] = (d["full_date"].dt.dayofweek >= 5).astype(int)
d["full_date"] = d["full_date"].dt.strftime("%Y-%m-%d")
d.to_sql("dim_date", conn, if_exists="append", index=False)

# ---- other dimensions (surrogate keys are generated by the database)
cust_ok[["customer_id", "name", "email", "city", "signup_date"]] \
    .to_sql("dim_customer", conn, if_exists="append", index=False)
prod_ok[["product_id", "product_name", "category", "unit_price", "unit_cost"]] \
    .to_sql("dim_product", conn, if_exists="append", index=False)
pay_ok[["method", "status"]].drop_duplicates() \
    .to_sql("dim_payment", conn, if_exists="append", index=False)

# ---- fact table: look up surrogate keys, compute measures
dc = pd.read_sql("SELECT customer_key, customer_id FROM dim_customer", conn)
dp = pd.read_sql("SELECT product_key, product_id, unit_price, unit_cost FROM dim_product", conn)
dpay = pd.read_sql("SELECT payment_key, method, status FROM dim_payment", conn)

f = (orders_ok.merge(pay_ok[["order_id", "amount", "method", "status"]], on="order_id")
    .merge(dc, on="customer_id")
    .merge(dp, on="product_id")
    .merge(dpay, on=["method", "status"]))
f["date_key"] = pd.to_datetime(f["order_date"]).dt.strftime("%Y%m%d").astype(int)
f["gross_amount"] = (f["quantity"] * f["unit_price"]).round(2)
f["discount_amount"] = (f["gross_amount"] * f["discount_pct"] / 100).round(2)
f["net_amount"] = (f["gross_amount"] - f["discount_amount"]).round(2)
f["cost_amount"] = (f["quantity"] * f["unit_cost"]).round(2)
f["profit"] = (f["net_amount"] - f["cost_amount"]).round(2)
f["amount_paid"] = f["amount"]

fact_cols = ["order_id", "date_key", "customer_key", "product_key", "payment_key", "quantity",
             "gross_amount", "discount_amount", "net_amount", "cost_amount", "profit", "amount_paid"]
f[fact_cols].to_sql("fact_sales", conn, if_exists="append", index=False)

conn.executescript("""
CREATE INDEX idx_fact_date ON fact_sales(date_key);
CREATE INDEX idx_fact_customer ON fact_sales(customer_key);
CREATE INDEX idx_fact_product ON fact_sales(product_key);
CREATE INDEX idx_fact_payment ON fact_sales(payment_key);

CREATE VIEW vw_sales_report AS
SELECT f.order_id, d.full_date, d.year, d.quarter, d.month, d.month_name, d.day_name, d.is_weekend,
       c.name AS customer, c.city, p.product_name, p.category, pay.method, pay.status,
       f.quantity, f.gross_amount, f.discount_amount, f.net_amount, f.cost_amount, f.profit
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_payment pay ON f.payment_key = pay.payment_key;
""")
conn.commit()

print("Warehouse table sizes:")

```

```
print(pd.read_sql("""
SELECT 'dim_date' AS tbl, COUNT(*) AS rows_ FROM dim_date UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product UNION ALL
SELECT 'dim_payment', COUNT(*) FROM dim_payment UNION ALL
SELECT 'fact_sales', COUNT(*) FROM fact_sales""", conn).to_string(index=False))
```

## Output

### STEP 4: DATA WAREHOUSE (star schema)

Warehouse table sizes:

| tbl          | rows |
|--------------|------|
| dim_date     | 365  |
| dim_customer | 30   |
| dim_product  | 12   |
| dim_payment  | 12   |
| fact_sales   | 200  |

## STEP 5: RECONCILIATION CHECKS

### Code

```
print("\n" + "=" * 60 + "\nSTEP 5: RECONCILIATION CHECKS\n" + "=" * 60)
fact_rows = conn.execute("SELECT COUNT(*) FROM fact_sales").fetchone()[0]
net_sum, paid_sum = conn.execute("SELECT SUM(net_amount), SUM(amount_paid) FROM fact_sales").fetchone()
orphans = conn.execute("""
SELECT COUNT(*) FROM fact_sales f
LEFT JOIN dim_date d ON f.date_key = d.date_key
LEFT JOIN dim_customer c ON f.customer_key = c.customer_key
LEFT JOIN dim_product p ON f.product_key = p.product_key
LEFT JOIN dim_payment pay ON f.payment_key = pay.payment_key
WHERE d.date_key IS NULL OR c.customer_key IS NULL
OR p.product_key IS NULL OR pay.payment_key IS NULL""").fetchone()[0]

print("Fact rows == valid orders : ", "PASS" if fact_rows == len(orders_ok) else "FAIL", f'({fact_rows} vs {len(orders_ok)})')
print("Sum(net_amount) == Sum(paid) : ", "PASS" if abs(net_sum - paid_sum) < 0.01 else "FAIL", f'({net_sum:,.2f} vs {paid_sum:,.2f})')
print("No orphan dimension keys in fact : ", "PASS" if orphans == 0 else "FAIL", f'({orphans} orphans)')
```

## Output

### STEP 5: RECONCILIATION CHECKS

Fact rows == valid orders : PASS (200 vs 200)  
Sum(net\_amount) == Sum(paid) : PASS (5,678,625.50 vs 5,678,625.50)  
No orphan dimension keys in fact : PASS (0 orphans)

## STEP 6: REPORTING (only SUCCESS payments count as revenue)

### Code

```
print("\n" + "=" * 60 + "\nSTEP 6: REPORTS\n" + "=" * 60)

kpi = pd.read_sql("""
SELECT COUNT(*) AS orders, SUM(quantity) AS units_sold,
ROUND(SUM(net_amount), 2) AS revenue, ROUND(AVG(net_amount), 2) AS avg_order_value,
ROUND(SUM(profit), 2) AS profit,
ROUND(100.0 * SUM(profit) / SUM(net_amount), 1) AS profit_margin_pct
FROM vw_sales_report WHERE status = 'SUCCESS'""", conn)
print("\nREPORT 1: Key performance indicators\n", kpi.to_string(index=False))

monthly = pd.read_sql("""
SELECT month, month_name, COUNT(*) AS orders, ROUND(SUM(net_amount), 2) AS revenue
FROM vw_sales_report WHERE status = 'SUCCESS'
GROUP BY month, month_name ORDER BY month""", conn)
print("\nREPORT 2: Monthly revenue\n", monthly.to_string(index=False))

category = pd.read_sql("""
SELECT category, SUM(quantity) AS units, ROUND(SUM(net_amount), 2) AS revenue,
ROUND(SUM(profit), 2) AS profit
FROM vw_sales_report WHERE status = 'SUCCESS'
```

```

GROUP BY category ORDER BY revenue DESC""", conn)
print("\nREPORT 3: Revenue by category\n", category.to_string(index=False))

city = pd.read_sql("""
SELECT city, COUNT(*) AS orders, ROUND(SUM(net_amount), 2) AS revenue
FROM vw_sales_report WHERE status = 'SUCCESS'
GROUP BY city ORDER BY revenue DESC""", conn)
print("\nREPORT 4: Revenue by city\n", city.to_string(index=False))

top5 = pd.read_sql("""
SELECT customer, city, COUNT(*) AS orders, ROUND(SUM(net_amount), 2) AS total_spent
FROM vw_sales_report WHERE status = 'SUCCESS'
GROUP BY customer, city ORDER BY total_spent DESC LIMIT 5""", conn)
print("\nREPORT 5: Top 5 customers\n", top5.to_string(index=False))

top_products = pd.read_sql("""
SELECT product_name, category, SUM(quantity) AS units, ROUND(SUM(net_amount), 2) AS revenue
FROM vw_sales_report WHERE status = 'SUCCESS'
GROUP BY product_name, category ORDER BY revenue DESC LIMIT 5""", conn)
print("\nREPORT 6: Top 5 products\n", top_products.to_string(index=False))

pay_summary = pd.read_sql("""
SELECT method, status, COUNT(*) AS payments, ROUND(SUM(net_amount), 2) AS amount
FROM vw_sales_report GROUP BY method, status ORDER BY method, status""", conn)
print("\nREPORT 7: Payment method x status\n", pay_summary.to_string(index=False))

weekend = pd.read_sql("""
SELECT CASE is_weekend WHEN 1 THEN 'Weekend' ELSE 'Weekday' END AS day_type,
COUNT(*) AS orders, ROUND(SUM(net_amount), 2) AS revenue
FROM vw_sales_report WHERE status = 'SUCCESS' GROUP BY is_weekend""", conn)
print("\nREPORT 8: Weekday vs weekend\n", weekend.to_string(index=False))

# ---- dashboard
fig, ax = plt.subplots(2, 2, figsize=(13, 9))
monthly.plot(x="month_name", y="revenue", kind="bar", ax=ax[0, 0], legend=False, title="Monthly Revenue")
category.plot(x="category", y="revenue", kind="bar", ax=ax[0, 1], legend=False, title="Revenue by Category", color="tab:green")
city.plot(x="city", y="revenue", kind="bar", ax=ax[1, 0], legend=False, title="Revenue by City", color="tab:orange")
top5.plot(x="customer", y="total_spent", kind="barh", ax=ax[1, 1], legend=False, title="Top 5 Customers", color="tab:purple")
ax[1, 1].invert_yaxis()
plt.tight_layout()
plt.savefig("mini_project/reports/dashboard.png", dpi=120)
plt.show()

# ---- export reports
for name, df in [("kpi", kpi), ("monthly_revenue", monthly), ("category_revenue", category),
                ("city_revenue", city), ("top_customers", top5), ("top_products", top_products),
                ("payment_summary", pay_summary), ("data_quality_summary", dq_df)]:
    df.to_csv(f"mini_project/reports/{name}.csv", index=False)

conn.close()
shutil.make_archive("mini_project_output", "zip", "mini_project")
print("\nAll outputs zipped to 'mini_project_output.zip' (database, raw files, reports, dashboard)")

```

## Output

### STEP 6: REPORTS

#### REPORT 1: Key performance indicators

| orders | units_sold | revenue   | avg_order_value | profit    | profit_margin_pct |
|--------|------------|-----------|-----------------|-----------|-------------------|
| 178    | 539        | 4840381.5 | 27193.15        | 1314026.5 | 27.1              |

#### REPORT 2: Monthly revenue

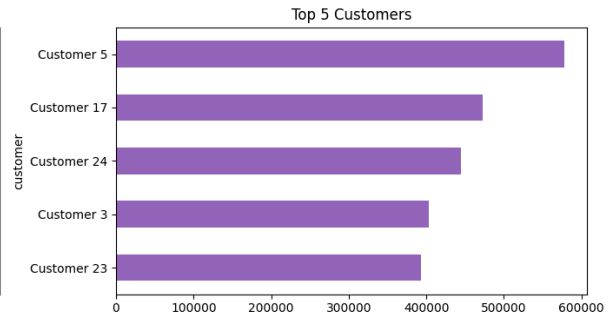
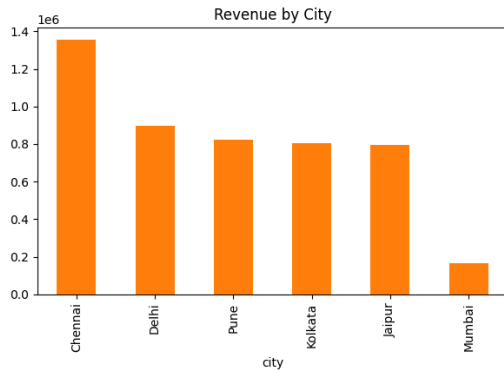
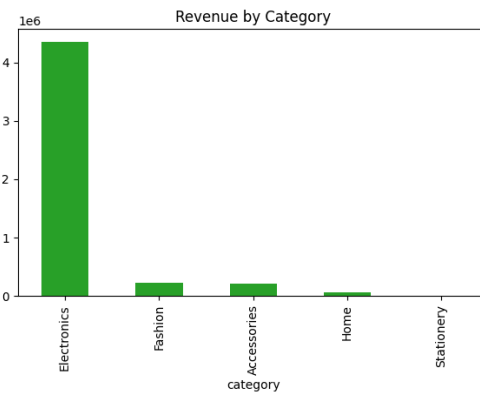
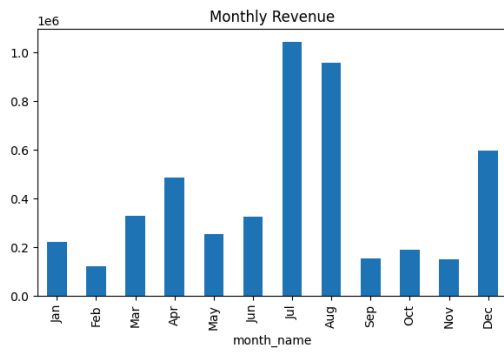
| month | month_name | orders | revenue   |
|-------|------------|--------|-----------|
| 1     | Jan        | 18     | 221164.0  |
| 2     | Feb        | 20     | 123952.5  |
| 3     | Mar        | 12     | 330370.0  |
| 4     | Apr        | 14     | 487401.0  |
| 5     | May        | 11     | 255630.0  |
| 6     | Jun        | 11     | 324255.0  |
| 7     | Jul        | 19     | 1044285.0 |
| 8     | Aug        | 19     | 958672.0  |
| 9     | Sep        | 11     | 153412.5  |
| 10    | Oct        | 14     | 191318.5  |
| 11    | Nov        | 15     | 152017.5  |

```

12    Dec    14 597903.5
REPORT 3: Revenue by category
  category units revenue  profit
Electronics 158 4353450.0 1184550.0
  Fashion   123 221280.0  57690.0
Accessories 123 206000.0  55150.0
  Home      100 56967.5  15912.5
Stationery   35  2684.0   724.0
REPORT 4: Revenue by city
  city orders revenue
Chennai   43 1353625.0
  Delhi    33 895800.5
  Pune     29 824087.5
Kolkata   39 803775.0
Jaipur    21 796437.5
Mumbai    13 166656.0
REPORT 5: Top 5 customers
  customer city orders total_spent
Customer 5  Delhi     7   577562.5
Customer 17 Chennai    9   472984.0
Customer 24 Jaipur     9   444485.0
Customer 3  Chennai    9   403610.0
Customer 23 Pune       7   393010.0
REPORT 6: Top 5 products
product_name category units revenue
Laptop Electronics 47 2472250.0
Smartphone Electronics 61 1298000.0
Monitor Electronics 50 583200.0
Headphones Accessories 54 129375.0
Sneakers Fashion 41 123360.0
REPORT 7: Payment method x status
method status payments amount
CARD FAILED 3 283925.0
CARD PENDING 4 30534.0
CARD SUCCESS 59 1664750.0
COD FAILED 2 12250.0
COD PENDING 1 5700.0
COD SUCCESS 34 866547.0
NETBANKING FAILED 2 114500.0
NETBANKING PENDING 1 10000.0
NETBANKING SUCCESS 33 568673.5
UPI FAILED 7 368075.0
UPI PENDING 2 13260.0
UPI SUCCESS 52 1740411.0
REPORT 8: Weekday vs weekend
day_type orders revenue
Weekday 134 3047638.0
Weekend 44 1792743.5

```

**Output (chart)**



## Output

All outputs zipped to 'mini\_project\_output.zip' (database, raw files, reports, dashboard)